

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий и искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторных работ №1 - 5

по дисциплине «Теория графов»

«Алгоритмы на графах»

Вариант 16

Студент группы 5130201/40001

_____ Денисенко Е.Д.

Преподаватель

_____ Востров А.В.

«___» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Математическое описание	6
1.1 Граф	6
1.2 Логарифмическое распределение №1	6
1.3 Эксцентриситет, центр и диаметр графа	8
1.4 Метод Шимбелла	9
1.5 Нахождение маршрута между двумя вершинами	9
1.6 Обход графа поиском в глубину	12
1.7 Алгоритм Дейкстры	12
1.8 Алгоритм Форда-Фалкерсона	13
1.9 Задача о нахождении заданного потока минимальной стоимости	15
1.10 Остов	17
1.11 Матричная теорема Кирхгофа	17
1.12 Алгоритм Прима	17
1.13 Кодирование Прюфера	18
1.14 Декодирование Прюфера	19
1.15 Минимальное рёберное покрытие	20
1.16 Эйлеров граф	20
1.17 Алгоритм Хирхольцера	20
1.18 Фундаментальная система разрезов	21
2 Особенности реализации	22
2.1 Класс RandomGenerator	22
2.2 Класс LogDistribution	22
2.3 Алгоритм логарифмического распределения	23
2.4 Класс Graph	24
2.4.1 Построение неориентированного графа по степеням вершин	24
2.4.2 Построение ориентированного графа по степеням вершин	27
2.5 Класс GraphAnalyzer	30
2.5.1 Вычисление эксцентриситетов	30
2.6 Класс WeightedGraph	31
2.6.1 Генерация весовой матрицы	31
2.6.2 Метод Шимбелла	32
2.6.3 Алгоритм Дейкстры	33
2.7 Класс PathFinder	34
2.8 Класс MaxFlowGraph	35
2.8.1 Алгоритм Форда-Фалкерсона	35
2.8.2 Поток минимальной стоимости	36
2.9 Класс Skeleton	38

2.9.1	Матричная теорема Кирхгофа	38
2.9.2	Алгоритм Прима	38
2.9.3	Кодирование и декодирование Прюфера	39
2.9.4	Минимальное рёберное покрытие остова	41
2.9.5	Минимальное рёберное покрытие графа	43
2.9.6	Фундаментальная система разрезов	45
2.10	Эйлеров цикл	48
2.11	Функция main и графический интерфейс	53
3	Результаты работы программы	54
3.1	Лабораторная работа №1	54
3.1.1	Генерация графа и характеристики	54
3.1.2	Генерация весовой матрицы	55
3.1.3	Метод Шимбелла	56
3.1.4	Поиск маршрутов	56
3.2	Лабораторная работа №2	57
3.2.1	Обход в глубину (DFS)	57
3.2.2	Алгоритм Дейкстры	57
3.2.3	Сравнение скоростей реализованных алгоритмов	58
3.3	Лабораторная работа №3	59
3.3.1	Максимальный поток (Форд-Фалкерсон)	59
3.3.2	Поток минимальной стоимости	60
3.4	Лабораторная работа №4	60
3.4.1	Число остовных деревьев (Кирхгоф)	60
3.4.2	Минимальный по весу остов (алгоритм Прима)	61
3.4.3	Код Прюфера	62
3.4.4	Минимальное рёберное покрытие	63
3.5	Лабораторная работа №5	64
3.5.1	Проверка эйлеровости и построение эйлерова цикла	64
3.5.2	Фундаментальная система разрезов	66
	Заключение	69
	Список использованной литературы	71

Введение

Отчёт состоит из описания 5 лабораторных работ. Каждая работа включает в себя реализацию различных алгоритмов на случайно сгенерированных связных ациклических графах. Генерация графов осуществляется с использованием логарифмического распределения №1 (параметры распределения задаются как константы и подбираются экспериментально).

ЛАБОРАТОРНАЯ РАБОТА № 1

1. Сформировать случайным образом связный ациклический ориентированный граф в соответствии с логарифмическим распределением с введенным пользователем количеством вершин. Генерация происходит по степеням вершин.

2. Посчитать эксцентриситеты вершин, выделить центр графа и диаметральные вершины.

3. Реализовать метод Шимбелла для сгенерированной с помощью заданного распределения весовой матрицы (пользователь вводит количество ребер), в ответе представить минимальный и максимальный путь (в виде матрицы). Весовая матрица генерируется также в соответствии с логарифмическим распределением №1. При генерации учитывается генерация только положительных, только отрицательных и смешанных значений (на выбор пользователя).

4. Определить возможность построения маршрута от одной заданной точки до другой (номер вершин вводит пользователь) и указать количество различных маршрутов.

5. Реализовать меню.

ЛАБОРАТОРНАЯ РАБОТА № 2

1. Для заданных графов (случайно сгенерированных в предыдущей работе) выполнить обход вершин графа поиском в глубину.

2. Сгенерировать весовую матрицу с положительными весами и найти кратчайший путь для двух выбранных пользователем вершин с помощью классического алгоритма Дейкстры. В результате выводится не только расстояние, но и сам путь в виде последовательности вершин, а также вектор расстояний.

3. Сравнить скорости работы реализованных алгоритмов (по количеству итераций).

ЛАБОРАТОРНАЯ РАБОТА № 3

1. На основе сгенерированного ориентированного графа получить случайным образом матрицы пропускных способностей и стоимости.

2. Для полученного графа найти максимальный поток по алгоритму Форда-Фалкерсона.

3. Вычислить заданный поток минимальной стоимости (в качестве величины потока брать значение, равное $\lfloor 2/3 \cdot max \rfloor$, где max – максимальный поток). Для этого использовать ранее реализованный алгоритм Дейкстры.

ЛАБОРАТОРНАЯ РАБОТА № 4

1. Для сгенерированного неориентированного графа найти число остовных деревьев, используя матричную теорему Кирхгофа.

2. Построить минимальный по весу остов для сгенерированного неориентированного взвешенного графа, используя алгоритм Прима. Полученный остов закодировать при помощи кода Прюфера и декодировать его. Сохранять веса при кодировании обязательно.

3. Реализовать на исходном графе и полученном остове (по выбору пользователя) алгоритм нахождения минимального реберного покрытия.

ЛАБОРАТОРНАЯ РАБОТА № 5

1. Для заданных неориентированных графов проверить, является ли граф эйлеровым. Если нет, то модифицировать граф (логировать, что изменено). Построить эйлеров цикл.

2. Для заданных неориентированных графов построить кратчайший остов (алгоритмом из четвертой работы). На основе данного остова получить фундаментальную систему разрезов, получение всех остальных разрезов на основе фундаментальных с помощью операции симметрической разности (выбранные разрезы вводит пользователь).

Лабораторные работы выполнены на языке C++ с использованием фреймворка Qt в среде разработки Qt Creator. Реализован графический интерфейс пользователя.

1 Математическое описание

1.1 Граф

Простой граф $G(V, E)$ есть совокупность двух множеств – непустого множества V и множества E неупорядоченных пар различных элементов множества V . Множество V называется множеством вершин, множество E называется множеством рёбер.

$$G(V, E) = \langle V, E \rangle, \quad V \neq \emptyset, \quad E \subseteq V \times V, \quad \{v, v\} \notin E, \quad v \in V,$$

то есть множество E состоит из 2-элементных подмножеств множества V .

Сопутствующие термины:

- Вершина графа G есть элемент множества вершин $v \in V(G)$;
- Ребро графа G есть элемент множества рёбер $e \in E(G)$, или $e = \{v_1, v_2\}$, где $v_1 \in V(G)$, $v_2 \in V(G)$;
- Элементами графа G называются его вершины $v \in V(G)$ и рёбра $e \in E(G)$ графа;
- Вершина v инцидентна ребру e , если $v \in e$; тогда ещё говорят, что e есть ребро при v ;
- Соседние (смежные) вершины есть такие вершины v_1 и v_2 , что $\{v_1, v_2\} \in E(G)$ или, другими словами, обе вершины являются конечными для одного ребра;
- Степень вершины $d(v)$ есть количество инцидентных ей рёбер.

1.2 Логарифмическое распределение №1

Логарифмическое распределение №1 – это дискретное вероятностное распределение с параметром p ($0 < p < 1$).

Для генерации случайных чисел по логарифмическому распределению используется блок-схема из справочника Вадзинского. Алгоритм основан на последовательном вычислении вероятностей $P(X = k)$ и сравнении с равномерно распределённой случайной величиной $r \sim U(0, 1)$. Блок-схема представлена на рисунке 1.

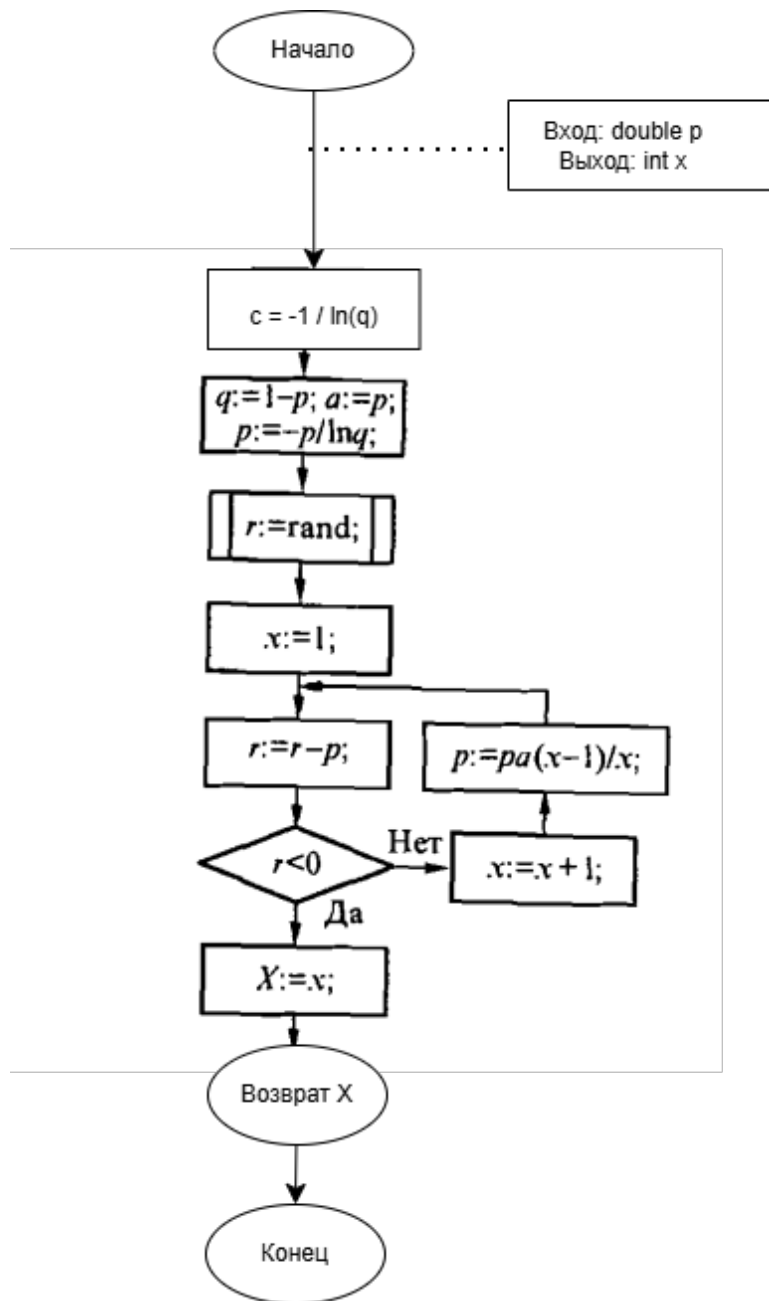


Рис. 1: Блок-схема логарифмического распределения №1

Обозначим $q = 1 - p$, $c = -1 / \ln(q)$. Начальное значение: $p(1) = -p / \ln(q)$. На каждом шаге вычитаем $p(x)$ из r , и если $r < 0$, то возвращаем x ; иначе увеличиваем x и пересчитываем $p(x + 1) = p(x) \cdot a \cdot (x - 1) / x$, где $a = p$.

Параметр p подбирается экспериментально в зависимости от требуемого среднего значения степени вершин. Экспериментально было выбрано значение $p = 0,7$. Гистограмма, приведённая в справочнике Вадзинского, представлена на рисунке 1.2. Полученная гистограмма для распределения 1000 значений представлена на рисунке 2.

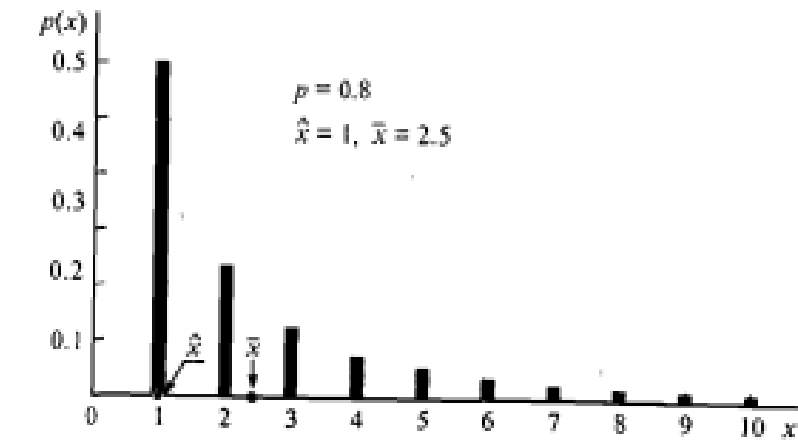


Рис. 2.18. Функция вероятности логарифмического распределения 1.

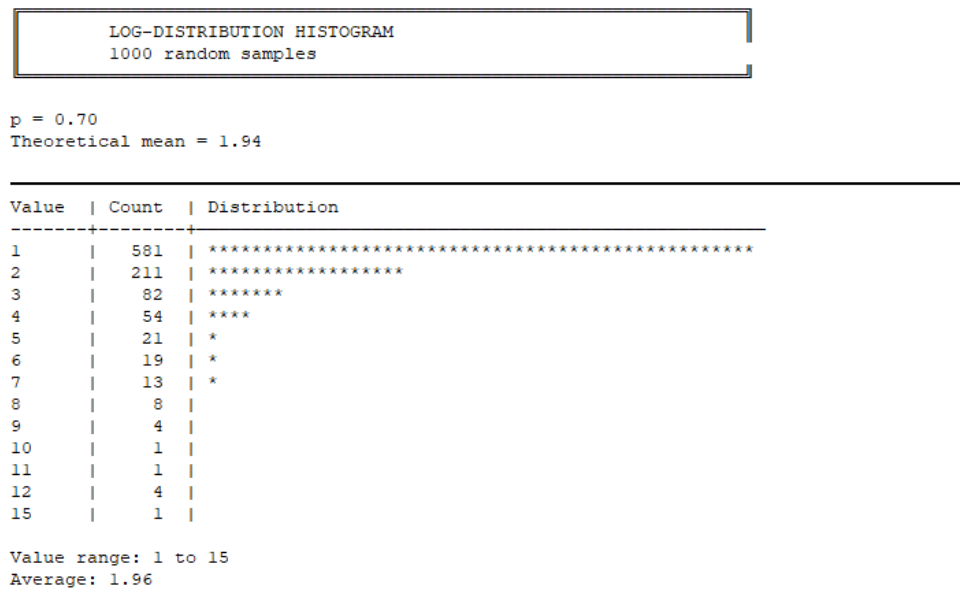


Рис. 2: Гистограмма для 1000 значений

1.3 Эксцентриситет, центр и диаметр графа

Эксцентриситет вершины v – это максимальное расстояние от v до любой другой вершины графа:

$$\varepsilon(v) = \max_{u \in V} d(v, u)$$

Радиус графа – это минимальный эксцентриситет:

$$r(G) = \min_{v \in V} \varepsilon(v)$$

Центр графа – это множество вершин, эксцентриситет которых равен радиусу:

$$C(G) = \{v \in V : \varepsilon(v) = r(G)\}$$

Диаметр графа – это максимальный эксцентриситет:

$$d(G) = \max_{v \in V} \varepsilon(v)$$

Диаметральные вершины – это вершины, эксцентриситет которых равен диаметру.

Для нахождения эксцентриситетов применяется обход в ширину (BFS) из каждой вершины, что позволяет вычислить все кратчайшие расстояния.

1.4 Метод Шимбелла

Метод Шимбелла позволяет находить минимальные и максимальные пути между вершинами, состоящие из заданного количества ребер.

- Операция умножения двух величин a и b при возведении матрицы в степень соответствует их алгебраической сумме, то есть:

$$a * b = b * a \rightarrow a + b = b + a$$

$$a * 0 = 0 * a = 0 \rightarrow a + 0 = 0$$

- Операция сложения двух величин a и b заменяется выбором из этих величин максимума или минимума, то есть:

$$a + b = b + a = \min(\max)\{a, b\}$$

Нули при этом игнорируются.

Сложность алгоритма: $O(p^3 * L)$, где p - количество вершин, L - заданная длина маршрута.

1.5 Нахождение маршрута между двумя вершинами

Для поиска всех простых путей между заданными вершинами s и t используется рекурсивный алгоритм поиска в глубину с возвратом.

Сложность алгоритма: $O(P * V)$, где P - число найденных простых путей.

Пусть $G = (V, E)$ – граф, заданный списками смежности $\Gamma[v]$. Требуется найти все простые пути из вершины s в вершину t .

Алгоритм рекурсивно обходит граф, начиная с вершины s , и строит все возможные пути, не содержащие повторяющихся вершин. При достижении целевой вершины t текущий путь сохраняется. После обработки всех соседей текущей вершины выполняется шаг возврата – текущая вершина исключается из пути, отметка о её посещении снимается, что позволяет использовать её в других ветвях поиска. Блок-схема алгоритма представлена на рисунке 3.

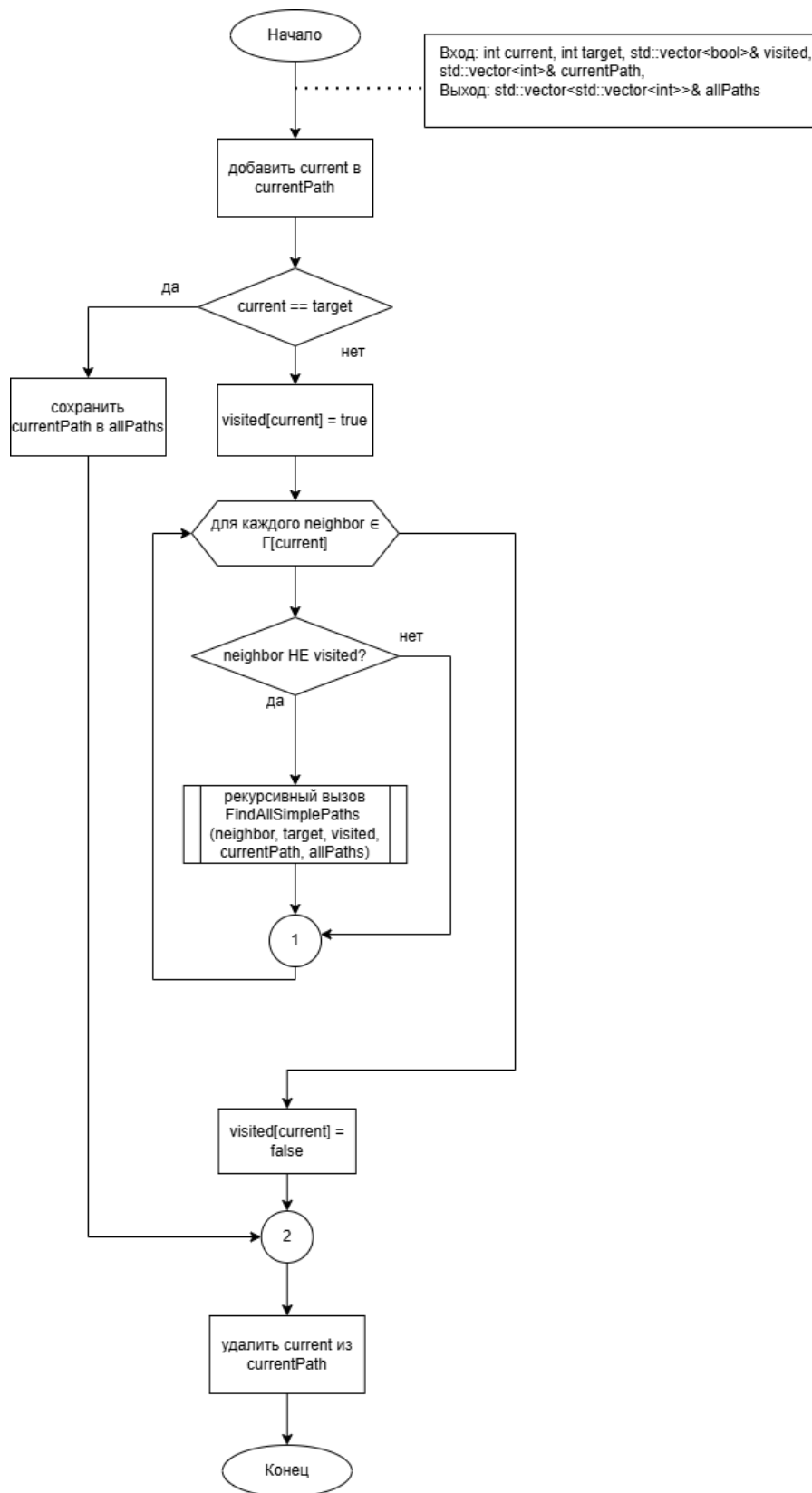


Рис. 3:

Блок-схема алгоритма нахождения маршрута между двумя вершинами

Перед поиском всех путей выполняется проверка существования хотя бы одного пути с помощью поиска в ширину. Сложность этой проверки составляет $O(|V| + |E|)$. Блок-схема алгоритма представлена на рисунке 4.

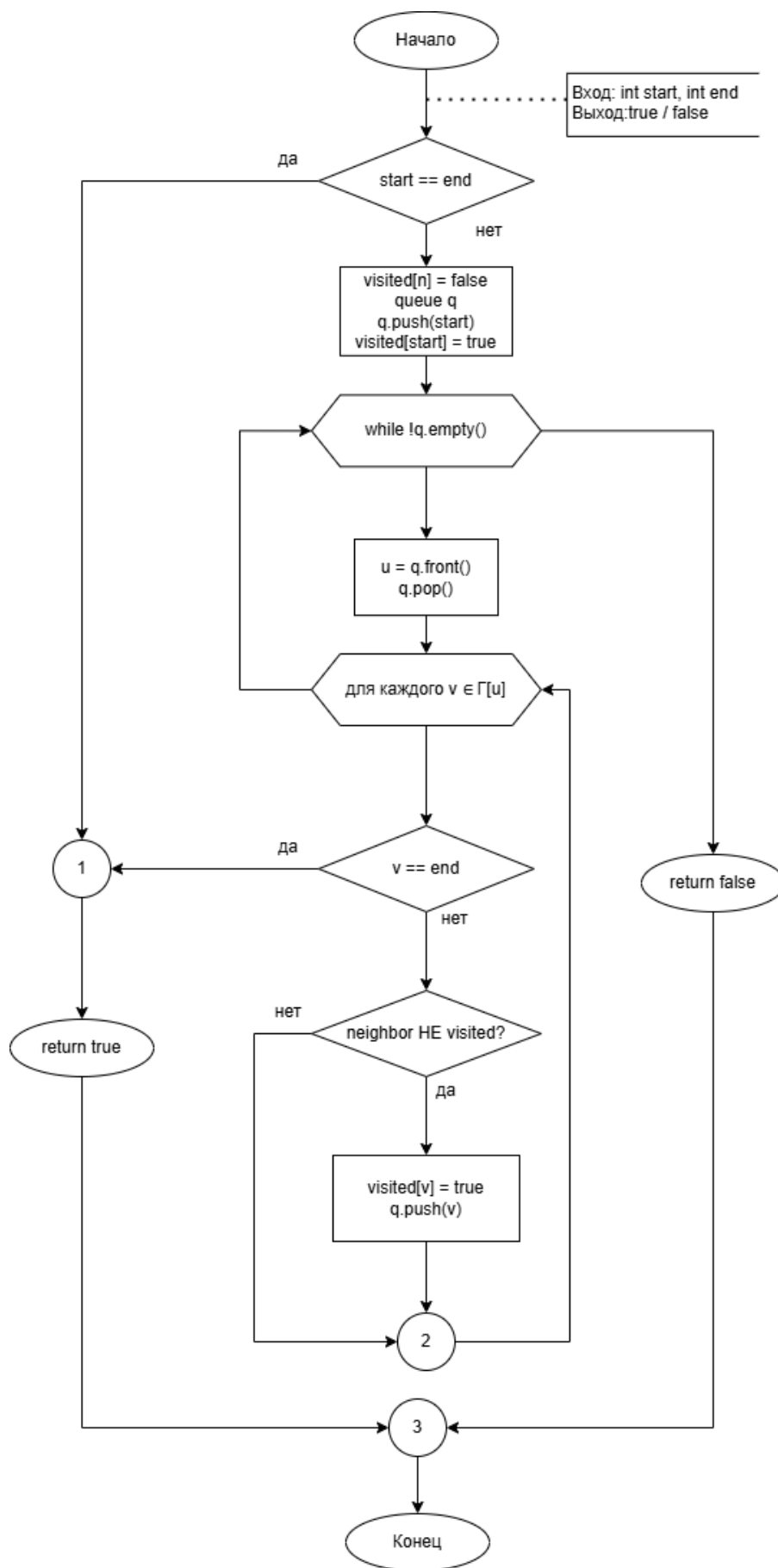


Рис. 4:

Блок-схема алгоритма проверки существования хотя бы одного пути между двумя вершинами

1.6 Обход графа поиском в глубину

Обход в глубину (DFS) – один из основных методов обхода графа, часто используемый для проверки связности, поиска цикла и компонент сильной связности.

Общая идея алгоритма состоит в следующем: для каждой не пройденной вершины необходимо найти все не пройденные смежные вершины и повторить поиск для них.

1. Выбираем любую вершину из еще не пройденных, обозначим ее как u ;
2. Запускаем процедуру $dfs(u)$:
 - Помечаем вершину u как пройденную;
 - Для каждой не пройденной смежной с u вершиной (назовем ее v) запускаем $dfs(v)$;
3. Повторяем шаги 1 и 2, пока все вершины не окажутся пройденными.

Время работы алгоритма оценивается как $O(V + E)$.

Псевдокод поиска в глубину приведен на рисунке 5

```
DFS(s):
  for all vertices  $v$  in topological order
    if  $v = s$ 
       $dist(v) \leftarrow 0$ 
    else
       $dist(v) \leftarrow \infty$ 
      for all edges  $u \rightarrow v$ 
        if  $dist(v) > dist(u) + w(u \rightarrow v)$        $\llcorner \text{if } u \rightarrow v \text{ is tense} \llcorner$ 
           $dist(v) \leftarrow dist(u) + w(u \rightarrow v)$    $\llcorner \text{relax } u \rightarrow v \llcorner$ 
```

Рис. 5: Псевдокод обхода графа в глубину

1.7 Алгоритм Дейкстры

Алгоритм Дейкстры – это алгоритм для вычисления кратчайших путей в графе с положительными весами ребер. Описание алгоритма:

- Инициализация:
 - Создается вектор расстояний T , где $T[i]$ – минимальное расстояние от начальной вершины до вершины i . Изначально все $T[i]$ равны бесконечности, кроме $T[start]$, которое равно 0.
 - Создается вектор предшественников H , где $H[i]$ – предыдущая вершина на кратчайшем пути к i .

- Создается вектор меток X , где $X[i] = 1$, если вершина уже обработана.
- На каждой итерации обновляются пометки всех соседей текущей вершины, затем выбирается ближайшая необработанная вершина.
- Процесс повторяется до тех пор, пока не будет достигнута конечная вершина или множество необработанных вершин не станет пустым.

Трудоемкость алгоритма – $O(V^2)$. Ограничение алгоритма: некорректно работает, если граф имеет ребра (дуги) отрицательного веса.

Псевдокод алгоритма Дейкстры представлен на рисунке 6.

```

Вход: орграф  $G(V, E)$ , заданный матрицей длин дуг  $C : \text{array}[1..p, 1..p] \text{ of real}$ ;
 $s$  и  $t$  – вершины графа.
Выход: векторы  $T : \text{array}[1..p] \text{ of real}$ ; и  $H : \text{array}[1..p] \text{ of } 0..p$ . Если вершина  $v$ 
лежит на кратчайшем пути от  $s$  к  $t$ , то  $T[v]$  – длина кратчайшего пути от  $s$  к
 $v$ ;  $H[v]$  – вершина, непосредственно предшествующая  $v$  на кратчайшем пути.
for  $v$  from 1 to  $p$  do
     $T[v] := \infty$  { кратчайший путь неизвестен }
     $X[v] := 0$  { все вершины не отмечены }
end for
 $H[s] := 0$  {  $s$  ничего не предшествует }
 $T[s] := 0$  { кратчайший путь имеет длину 0... }
 $X[s] := 1$  { ... и он известен }
 $v := s$  { текущая вершина }
 $M$ : { обновление пометок }
for  $u \in \Gamma(v)$  do
    if  $X[u] = 0$  &  $T[u] > T[v] + C[v, u]$  then
         $T[u] := T[v] + C[v, u]$  { найден более короткий путь из  $s$  в  $u$  через  $v$  }
         $H[u] := v$  { запоминаем его }
    end if
end for
 $m := \infty$ ;  $v := 0$ 
{ поиск конца кратчайшего пути }
for  $u$  from 1 to  $p$  do
    if  $X[u] = 0$  &  $T[u] < m$  then
         $v := u$ ;  $m := T[u]$  { вершина  $v$  заканчивает кратчайший путь из  $s$  }
    end if
end for
if  $v = 0$  then
    stop { нет пути из  $s$  в  $t$  }
end if
if  $v = t$  then
    stop { найден кратчайший путь из  $s$  в  $t$  }
end if
 $X[v] := 1$  { найден кратчайший путь из  $s$  в  $v$  }
goto  $M$ 

```

Рис. 6: Псевдокод алгоритма Дейкстры

1.8 Алгоритм Форда-Фалкерсона

Алгоритм Форда-Фалкерсона решает задачу нахождения максимального потока в сети. Идея алгоритма заключается в следующем:

- Изначально величине потока присваивается значение 0 для всех u и v в транспортной сети.
- Затем величина потока итеративно увеличивается посредством поиска увеличивающего пути от источника s к стоку t .
- Процесс повторяется, пока можно найти увеличивающий путь.

Алгоритм расстановки пометок используется для поиска увеличивающих путей. Каждая вершина помечается тройкой: знак направления дуги ($\pm$), предыдущая вершина и величина изменения потока δ . Множество S хранит достижимые вершины, множество N – просмотренные.

Сложность алгоритма $O(p * q * U)$, где U – наибольшая величина максимальной пропускной способности сети, p – количество вершин, q – количество ребер.

Псевдокод алгоритма Форда-Фалкерсона представлен на рисунках 7 и 8.

Вход: сеть $G(V, E)$ с источником s и стоком t , заданная матрица пропускных способностей $C : \text{array } [1..p, 1..p] \text{ of real}$.
Выход: матрица максимального потока $F : \text{array } [1..p, 1..p] \text{ of real}$.

```

for  $u, v \in V$  do
   $F[u, v] := 0$  { вначале поток нулевой }
end for
 $M : \{ \text{итерация увеличения потока} \}$ 
for  $v \in V$  do
   $S[v] := 0; N[v] := 0; P[v] := (+, \text{nil}, 0)$  { инициализация }
end for
 $S[s] := 1; P[s] := (+, \text{nil}, \infty)$  { так как  $s \in S$  }
repeat
   $a := 0$  { признак расширения  $S$  }
  for  $v \in V$  do
    if  $S[v] = 1 \ \& \ N[v] = 0$  then
      for  $u \in \Gamma(v)$  do
        if  $S[u] = 0 \ \& \ F[v, u] < C[v, u]$  then
           $S[u] := 1; P[u] := (+, v, \min(P[v].\delta, C[v, u] - F[v, u])); a := 1$ 
        end if
      end for
    end if
  end for
   $N[v] := 1$  { узел  $v$  отмечен }
end if
end for
if  $S[t]$  then
   $x := t$  { текущий узел аугментальной цепи }
   $\delta := P[t].\delta$  { величина изменения потока }
  while  $x \neq s$  do
    if  $P[x].s = +$  then
       $F[P[x].n, x] := F[P[x].n, x] + \delta$  { увеличение потока }
    else
       $F[x, P[x].n] := F[x, P[x].n] - \delta$  { увеличение потока }
    end if
     $x := P[x].n$  { предыдущий узел аугментальной цепи }
  end while
  goto  $M$ 
end if
until  $a = 0$ 

```

Рис. 7: Псевдокод алгоритма Форда-Фалкерсона

```

end for
for  $u \in \Gamma^{-1}(v)$  do
  if  $S[u] = 0 \ \& \ F[u, v] > 0$  then
     $S[u] := 1; P[u] := (-, v, \min(P[v].\delta, F[u, v])); a := 1$ 
  end if
end for
 $N[v] := 1$  { узел  $v$  отмечен }
end if
end for
if  $S[t]$  then
   $x := t$  { текущий узел аугментальной цепи }
   $\delta := P[t].\delta$  { величина изменения потока }
  while  $x \neq s$  do
    if  $P[x].s = +$  then
       $F[P[x].n, x] := F[P[x].n, x] + \delta$  { увеличение потока }
    else
       $F[x, P[x].n] := F[x, P[x].n] - \delta$  { увеличение потока }
    end if
     $x := P[x].n$  { предыдущий узел аугментальной цепи }
  end while
  goto  $M$ 
end if
until  $a = 0$ 

```

Рис. 8: Псевдокод алгоритма Форда-Фалкерсона

1.9 Задача о нахождении заданного потока минимальной стоимости

Рассмотрим задачу определения потока заданной величины θ от s к t в сети $G = (S, U, \Omega, D)$, в которой каждая дуга (x_i, x_j) характеризуется пропускной способностью $d(x_i, x_j) \in \Omega$ и неотрицательной стоимостью

$d(x_i, x_j) \in D$ пересылки единичного потока из x_i в x_j вдоль дуги (x_i, x_j) .

Сложность алгоритма: $O(\theta * V^2)$.

Если $\theta > \varphi_{\max}$, где $\varphi_{\max}$ — величина максимального потока в сети G от s к t , то решения нет. Если же $\theta \leq \varphi_{\max}$, то может быть определено несколько различных потоков величины θ от s к t . Математическая модель задачи выглядит следующим образом. Минимизировать

$$\sum_{(x_i, x_j) \in U} d(x_i, x_j) \cdot \varphi(x_i, x_j),$$

где $\varphi(x_i, x_j)$ — поток по дуге (x_i, x_j) при ограничениях:

1. $0 \leq \varphi(x_i, x_j) \leq d(x_i, x_j)$ для $\forall (x_i, x_j) \in U$;
2. для начальной вершины $s \in S$: $\sum_{x_i \in S} \varphi(s, x_i) - \sum_{x_i \in S} \varphi(x_i, s) = \theta$ — уравнение истока;
3. для конечной вершины $t \in S$: $\sum_{x_i \in S} \varphi(t, x_i) - \sum_{x_i \in S} \varphi(x_i, t) = -\theta$ — уравнение стока;
4. для любой другой вершины $x_i \neq s, t$: $\sum_{x_j \in S} \varphi(x_i, x_j) - \sum_{x_j \in S} \varphi(x_j, x_i) = 0$ — условие сохранения потока.

Алгоритм решения задачи основан на последовательном нахождении путей минимальной стоимости в модифицированной сети. Модифицированная относительно данного потока φ сеть $\tilde{G} = (S, \tilde{U}, \tilde{\Omega}, \tilde{D})$ строится следующим образом:

1. $\tilde{S} = S$, то есть число и набор вершин в модифицированной сети не изменяется.
2. $\tilde{U} = U \cup V$, где V — некоторое множество фиктивных дуг. Таким образом, после модификации число дуг сети увеличивается.
3. Если $(x_j, x_i) \in S$ и $\varphi(x_j, x_i) > 0$, то дуга (x_i, x_j) включается в множество V . При этом полагается $\tilde{c}(x_i, x_j) = \varphi(x_j, x_i)$, $\tilde{d}(x_i, x_j) = -d(x_j, x_i)$.
4. Для всех ненасыщенных дуг, где нет противоположного потока, то есть для $(x_i, x_j) \in S$, $\varphi(x_i, x_j) < d(x_i, x_j)$ и $\varphi(x_j, x_i) = 0$ полагают $\tilde{c}(x_i, x_j) = d(x_i, x_j) - \varphi(x_j, x_i)$, $\tilde{d}(x_i, x_j) = d(x_i, x_j)$.
5. Для всех насыщенных дуг $(x_i, x_j) \in S$, $\varphi(x_i, x_j) = d(x_i, x_j)$ и $\varphi(x_j, x_i) = 0$ полагают $\tilde{c}(x_i, x_j) = 0$, $\tilde{d}(x_i, x_j) = \infty$.

1.10 Остов

Пусть $G = (V, E)$ – граф. Остовный подграф графа G – подграф, содержащий все вершины. Остовный подграф, являющийся деревом, называется остовом. Несвязный граф не имеет остова. Связный граф может иметь много остовов.

Минимальное остовное дерево графа G – остовное дерево $T = (V, E_T)$ графа G , сумма весов ребер которого минимальна.

1.11 Матричная теорема Кирхгофа

• Матрица Кирхгофа

Матрица B графа G размера $n \times n$ определяется следующим образом:

$$B[i, j] = \begin{cases} -1, & \text{если вершины с номерами } i \text{ и } j \text{ смежны;} \\ 0, & \text{если } i \neq j \text{ и вершины не смежны;} \\ \deg(v_i), & \text{если } i = j. \end{cases}$$

• Свойства матриц Кирхгофа

1. Суммы элементов в каждой строке и каждом столбце матрицы равны 0.
2. Алгебраические дополнения всех элементов матрицы равны между собой.
3. Определитель матрицы Кирхгофа равен нулю.

• Матричная теорема Кирхгофа

Число остовных деревьев в связном графе G порядка $n \geq 2$ равно алгебраическому дополнению любого элемента матрицы Кирхгофа B графа G .

1.12 Алгоритм Прима

Алгоритм Прима – это алгоритм построения минимального остовного дерева взвешенного связного неориентированного графа.

Алгоритм работает следующим образом:

1. Выбирается произвольная вершина u . Множество $S = \{u\}$, множество рёбер остова $T = \emptyset$.
2. Для каждой вершины $v \notin S$ запоминается ближайшая вершина остова $\alpha[v]$ и расстояние до нее $\beta[v]$.

3. На каждом шаге выбирается вершина $w \notin S$ с минимальным $\beta[w]$, добавляется в S , ребро $(\alpha[w], w)$ добавляется в T .
4. Обновляются метки α и β для соседей w .
5. Рост дерева продолжается до тех пор, пока не будут исчерпаны все вершины.

Общая сложность алгоритма составляет $O(V^2)$.

Псевдокод алгоритма Прима представлен на рисунках 9 и 10.

Вход: граф $G(V, E)$, заданный матрицей длин рёбер C .
Выход: множество T рёбер кратчайшего остова.

```

select  $u \in V$  { выбираем произвольную вершину }
 $S := \{u\}$  {  $S$  — множество вершин, включённых в кратчайший остов }
 $T := \emptyset$  {  $T$  — множество рёбер, включённых в кратчайший остов }
for  $v \in V - u$  do
  if  $v \in \Gamma(u)$  then
     $\alpha[v] := u$  {  $u$  — ближайшая вершина остова }
     $\beta[v] := C[u, v]$  {  $C[u, v]$  — длина соответствующего ребра }
  else
     $\alpha[v] := 0$  { ближайшая вершина остова неизвестна }
     $\beta[v] := \infty$  { и расстояние также неизвестно }
  end if
end for
for  $i$  from 1 to  $p - 1$  do

```

Рис. 9: Псевдокод алгоритма Прима

```

 $x := \infty$  { начальное значение для поиска ближайшей вершины }
for  $v \in V \setminus S$  do
  if  $\beta[v] < x$  then
     $w := v$  { нашли более близкую вершину }
     $x := \beta[v]$  { и расстояние до неё }
  end if
end for
 $S := S + w$  { добавляем найденную вершину в остов }
 $T := T + (\alpha[w], w)$  { добавляем найденное ребро в остов }
for  $v \in \Gamma(w)$  do
  if  $v \notin S$  then
    if  $\beta[v] > C[v, w]$  then
       $\alpha[v] := w$  { изменяем ближайшую вершину остова }
       $\beta[v] := C[v, w]$  { и длину ведущего к ней ребра }
    end if
  end if
end for
end for

```

Рис. 10: Псевдокод алгоритма Прима

1.13 Кодирование Прюфера

Алгоритм кодирования Прюфера состоит из следующих шагов:

Пока количество вершин больше двух:

- Выбираем лист v с минимальным номером.
- В код Прюфера добавляем номер вершины, смежной с v .
- Вершина v и инцидентное ей ребро удаляются из дерева.

Полученная последовательность длины $n - 2$ называется кодом Прюфера. Псевдокод для построения кода Прюфера представлен на рисунке 11. Сложность алгоритма: $O(N)$, где N - количество вершин остова.

```
Вход: Дерево  $T(V, E)$  в любом представлении, вершины дерева занумерованы  
числами  $1..p$  произвольным образом.  
Выход: Массив  $A$  : array  $[1..p - 1]$  of  $1..p$  — код Прюфера дерева  $T$ .  
for  $i$  from 1 to  $p - 1$  do  
   $v := \min \{k \in V \mid d(k) = 1\}$  { выбираем вершину  $v$  — висющую вершину с  
  наименьшим номером }  
   $A[i] := \Gamma(v)$  { заносим в код номер единственной вершины, смежной с  $v$  }  
   $V := V - v$  { удаляем вершину  $v$  из дерева }  
end for
```

Рис. 11: Код Прюфера

1.14 Декодирование Прюфера

Декодирование: имея код длины $n - 2$ и множество всех вершин $\{0, \dots, n - 1\}$, на каждом шаге берётся первый элемент кода и находится наименьшая вершина, не входящая в оставшуюся часть кода. Найденная вершина и текущий элемент кода образуют ребро дерева. Обе вершины удаляются из соответствующих массивов. После обработки всего кода оставшиеся две вершины образуют последнее ребро.

Сложность алгоритма: $O(N)$, где N - количество вершин остова. Псевдокод для распаковки кода Прюфера представлен на рисунке 12.

```
Вход: Массив  $A$  : array  $[1..p - 1]$  of  $1..p$  — код Прюфера дерева  $T$ .  
Выход: Дерево  $T(V, E)$ , заданное множеством рёбер  $E$ , вершины дерева  
занумерованы числами  $1..p$ .  
 $E := \emptyset$  { в начале множество рёбер пусто }  
 $B := 1..p$  { множество неиспользованных номеров вершин }  
for  $i$  from 1 to  $p - 1$  do  
   $v := \min \{k \in B \mid \forall j \geq i (k \neq A[j])\}$  { выбираем вершину  $v$  — неиспользо-  
  ванную вершину с наименьшим номером, который не встречается в остатке кода  
  Прюфера }  
   $E := E + (v, A[i])$  { добавляем ребро  $(v, A[i])$  }  
   $B := B - v$  { удаляем вершину  $v$  из списка неиспользованных }  
end for
```

Рис. 12: Декодирование Прюфера

1.15 Минимальное рёберное покрытие

Рёберное покрытие графа $G(V, E)$ – это множество рёбер $F \subseteq E$ такое, что каждая вершина графа инцидентна хотя бы одному ребру из F .

Минимальное рёберное покрытие – это рёберное покрытие с наименьшим числом рёбер. По второму тождеству Галлаи:

$$\nu(G) + \rho(G) = |V|,$$

где $\nu(G)$ – размер максимального паросочетания, $\rho(G)$ – число рёберного покрытия, $|V|$ – мощность множества вершин графа.

Для нахождения минимального рёберного покрытия сначала строится максимальное по размеру множество рёбер, не имеющих общих вершин, затем оставшиеся непокрытые вершины инцидентно покрываются дополнительными рёбрами минимального веса.

1.16 Эйлеров граф

Эйлеров граф – это граф, в котором существует эйлеров цикл, то есть замкнутый путь, проходящий по всем рёбрам графа ровно один раз.

Граф является эйлеровым тогда и только тогда, когда он связан и степени всех его вершин чётны.

Для поиска эйлерова цикла используется алгоритм на основе обхода в глубину:

- Выбрать произвольную вершину и поместить её в стек.
- Пока стек не пуст:
 - Если у верхней вершины есть непосещённые соседи, выбрать одного и поместить в стек, удалив соответствующее ребро.
 - Если непосещённых соседей нет – извлечь вершину из стека и добавить в результат.

Сложность: $O(V + E)$.

Если граф не является эйлеровым, его необходимо модифицировать: добавив или удалить рёбра так, чтобы степени всех вершин стали чётными. Модификация логируется.

1.17 Алгоритм Хирхольцера

Алгоритм Хирхольцера находит эйлеров цикл в неориентированном графе.

Перед запуском проверяется, что граф связан и все его вершины имеют чётную степень. Создаётся рабочая копия списков смежности $\Gamma[v]$, которая будет

модифицироваться в процессе работы. Исходный граф остаётся неизменным. Начиная с произвольной вершины v , алгоритм строит путь, добавляя на каждом шаге любое непройденное ребро и помещая пройденную вершину в стек S . Пройденное ребро удаляется из $\Gamma[v]$ и $\Gamma[u]$. Когда для вершины на вершине стека не остаётся инцидентных рёбер, она извлекается из стека и записывается в результирующую последовательность. Затем алгоритм возвращается к предыдущей вершине в стеке и продолжает обход. Результирующая последовательность формируется в порядке выхода вершин из стека, что соответствует обратному порядку относительно обхода.

Сложность алгоритма: $O(|E|)$.

Псевдокод алгоритма представлен на рисунке 13

```
function findEulerPath(v): // если граф является полуэйлеровым, то алгоритм следует запускать из вершины нечетной степени
    for u : u ∈ V
        if deg(u) mod 2 == 1
            v = u
            break
    S.push(v) // S – стек
    while not S.empty()
        w = S.top()
        found_edge = False
        for u : u ∈ V
            if (w, u) ∈ E // нашли ребро, по которому ещё не прошли
                S.push(u) // добавили новую вершину в стек
                E.remove(w, u)
                found_edge = True
                break
        if not found_edge
            S.pop() // не нашлось инцидентных вершине w рёбер, по которым ещё не прошли
            print(w)
```

Рис. 13: Псевдокод алгоритма Хирхольцера

1.18 Фундаментальная система разрезов

Разрез графа $G = (V, E)$ – это разбиение множества вершин V на два непустых подмножества V_1 и V_2 ($V_1 \cup V_2 = V$, $V_1 \cap V_2 = \emptyset$). Множество рёбер разреза – это множество рёбер с концами в разных частях.

Фундаментальная система разрезов строится по остовному дереву T : для каждого ребра $e \in T$ его удаление разбивает дерево на две компоненты, определяющие разрез S_e . Коциклический ранг графа равен $n - 1$ (числу рёбер остова).

Любой другой разрез можно получить как симметрическую разность фундаментальных разрезов:

$$S = S_{e_1} \Delta S_{e_2} \Delta \dots \Delta S_{e_k}$$

2 Особенности реализации

Программа реализована на языке C++ с использованием фреймворка Qt. Код разделен на несколько классов: Graph (структура графа), GraphAnalyzer (анализ характеристик), WeightedGraph (взвешенный граф и алгоритмы), PathFinder (поиск маршрутов), MaxFlowGraph (потoki), Skeleton (остовы и покрытия), LogDistribution (логарифмическое распределение), RandomGenerator (генератор случайных чисел), MainWindow (графический интерфейс).

2.1 Класс RandomGenerator

Класс RandomGenerator использует генератор псевдослучайных чисел `std::mt19937`. Предоставляет методы для генерации равномерно распределённых вещественных чисел в $[0, 1)$ и целых чисел в заданном диапазоне.

```
1 class RandomGenerator
2 {
3 private:
4     std::mt19937 gen;
5 public:
6     RandomGenerator();
7     double uniform();
8     int uniformInt(int min, int max);
9 };
10
11 RandomGenerator::RandomGenerator()
12     : gen(std::random_device{}()) {}
13
14 double RandomGenerator::uniform()
15 {
16     std::uniform_real_distribution<double> dist(0.0, 1.0);
17     return dist(gen);
18 }
19
20 int RandomGenerator::uniformInt(int min, int max)
21 {
22     std::uniform_int_distribution<int> dist(min, max);
23     return dist(gen);
24 }
25
```

2.2 Класс LogDistribution

Класс LogDistribution определяет константы и функции для генерации случайных чисел по логарифмическому распределению №1.

```
1 class LogDistribution
2 {
3 private:
```

```

4     double p;
5     double q;    // = 1 - p
6     double c;    // = -1 / ln(q)
7     RandomGenerator& rng;
8 public:
9     LogDistribution(double paramP, RandomGenerator& generator);
10    int generate();
11    double getMean() const;
12 };
13
14 LogDistribution::LogDistribution(double paramP, RandomGenerator& generator)
15     : p(paramP), rng(generator)
16 {
17     q = 1.0 - p;
18     c = -1.0 / std::log(q);
19 }
20

```

2.3 Алгоритм логарифмического распределения

Функция `generate` реализует генерацию случайных чисел по логарифмическому распределению №1. Используется блок-схема из справочника Вадзинского. Параметр p задаётся как константа.

Вход: *double* p - параметр распределения

Выход: *int* x - случайное натуральное число, соответствующее логарифмическому распределению №1

```

1     int LogDistribution::generate()
2     {
3         double a = p;
4         double currentP = -p / std::log(q);
5         double r = rng.uniform();
6         int x = 1;
7
8         while (true)
9         {
10            r -= currentP;
11            if (r < 0)
12            {
13                return x;
14            }
15            x++;
16            currentP = currentP * a * (x - 1) / x;
17        }
18    }

```

2.4 Класс Graph

Класс Graph представляет структуру графа (ориентированного или неориентированного). Хранит список смежности, степени вершин. Поддерживает построение графа по заданным степеням вершин с последующей проверкой связности. В секции private хранятся следующие атрибуты:

```
1 private:
2   int n; // количество вершин
3   GraphType type; // тип графа
4   std::vector<std::vector<int>> adj; // список смежности
5   std::vector<int> degrees; // исходные степени для(
неориент.)
6   std::vector<int> inDegrees; // входящие степени для(
ориент.)
7   std::vector<int> outDegrees; // исходящие степени для(
ориент.)
8   mutable int dfsIterations; // количество итераций для
поиска в глубину
9
```

2.4.1 Построение неориентированного графа по степеням вершин

Функция buildGraphFromDesiredDegrees строит неориентированный граф по заданному массиву желаемых степеней вершин. Каждой вершине назначается количество соединений, равной степени связи, затем точки перемешиваются и соединяются случайным способом. Петли и кратные рёбра игнорируются. Если граф получился несвязным, добавляются рёбра между компонентами.

Вход: *const std::vector<int> &desiredDegrees* - вектор желаемых степеней вершин

Выход: *vector<vector<int>> adj* - список смежности

```
1 bool Graph::buildGraphFromDesiredDegrees(const std::vector<int>&
desiredDegrees)
2 {
3   if (type == DIRECTED) {
4     return false;
5   }
6
7   int n = desiredDegrees.size();
8
9   std::vector<int> finalDegrees = desiredDegrees;
10
11   // нормировочка
12   int maxAllowed = n - 1;
13   int maxDegree = 0;
14   for (int d : finalDegrees) {
15     if (d > maxDegree) maxDegree = d;
16   }
```



```

17
18     if (maxDegree > maxAllowed) {
19         double coeff = (double)maxAllowed / maxDegree;
20         for (int i = 0; i < n; i++) {
21             finalDegrees[i] = (int)(finalDegrees[i] * coeff);
22             if (finalDegrees[i] < 1) finalDegrees[i] = 1;
23             if (finalDegrees[i] > maxAllowed) finalDegrees[i] = maxAllowed;
24         }
25     }
26
27     int sum = 0;
28     for (int d : finalDegrees) sum += d;
29
30     if (sum % 2 != 0) {
31         for (int i = 0; i < n; i++) {
32             if (finalDegrees[i] < n - 1) {
33                 finalDegrees[i]++;
34                 sum++;
35                 break;
36             }
37         }
38         if (sum % 2 != 0) {
39             finalDegrees[0]--;
40         }
41     }
42
43     degrees = finalDegrees;
44
45     for (auto& neighbors : adj)
46         neighbors.clear();
47
48     std::vector<int> stubs;
49     for (int i = 0; i < n; i++) {
50         for (int j = 0; j < degrees[i]; j++) {
51             stubs.push_back(i);
52         }
53     }
54
55     std::random_device rd;
56     std::mt19937 g(rd());
57     std::shuffle(stubs.begin(), stubs.end(), g);
58
59     std::set<std::pair<int, int>> edgesSet;
60
61     for (size_t i = 0; i + 1 < stubs.size(); i += 2) {
62         int u = stubs[i];
63         int v = stubs[i + 1];
64
65         if (u == v) continue;
66
67         if (u > v) std::swap(u, v);
68

```

```

69         if (edgesSet.find({u, v}) != edgesSet.end()) {
70             continue;
71         }
72
73         adj[u].push_back(v);
74         adj[v].push_back(u);
75         edgesSet.insert({u, v});
76     }
77
78     std::uniform_int_distribution<int> dist(0, n - 1);
79
80     while ((int)edgesSet.size() < sum / 2) {
81         int u = dist(g);
82         int v = dist(g);
83
84         if (u == v) continue;
85         if (u > v) std::swap(u, v);
86
87         if (edgesSet.find({u, v}) == edgesSet.end()) {
88             adj[u].push_back(v);
89             adj[v].push_back(u);
90             edgesSet.insert({u, v});
91         }
92     }
93
94     if (!isConnected()) {
95         std::vector<int> component(n, -1);
96         int compCount = 0;
97
98         for (int i = 0; i < n; i++) {
99             if (component[i] == -1) {
100                 std::queue<int> q;
101                 q.push(i);
102                 component[i] = compCount;
103
104                 while (!q.empty()) {
105                     int u = q.front();
106                     q.pop();
107                     for (int v : adj[u]) {
108                         if (component[v] == -1) {
109                             component[v] = compCount;
110                             q.push(v);
111                         }
112                     }
113                 }
114                 compCount++;
115             }
116         }
117
118         for (int c = 1; c < compCount; c++) {
119             int u = -1, v = -1;
120             for (int i = 0; i < n; i++) {

```

```

121         if (component[i] == 0 && u == -1) u = i;
122         if (component[i] == c && v == -1) v = i;
123     }
124     if (u != -1 && v != -1) {
125         adj[u].push_back(v);
126         adj[v].push_back(u);
127     }
128 }
129 }
130
131 for (int i = 0; i < n; i++) {
132     degrees[i] = adj[i].size();
133 }
134
135 return true;
136 }
137

```

2.4.2 Построение ориентированного графа по степеням вершин

Функция `buildDirectedGraphFromDesiredDegrees` строит ориентированный граф по заданному распределению входящих и исходящих степеней связи. Выполняется нормировка степеней, затем создаются полурёбра для исходящих связей, которые соединяются случайным образом с подходящими вершинами с приоритетом на вершины с большим количеством необходимых входящих степеней.

Вход: *const std::vector<int> &desiredOutDegrees* - вектор желаемых исходящих степеней вершин,

const std::vector<int> &desiredInDegrees - вектор желаемых входящих степеней вершин

Выход: *vector<vector<int>> adj* - список смежности

```

1 bool Graph::buildDirectedGraphFromDesiredDegrees(const std::vector<int>&
2     desiredOutDegrees,
3     const std::vector<int>& desiredInDegrees)
4 {
5     if (type != DIRECTED) return false;
6
7     int n = desiredOutDegrees.size();
8
9     std::vector<int> finalOutDegrees = desiredOutDegrees;
10    std::vector<int> finalInDegrees = desiredInDegrees;
11
12    // нормировочка
13    int maxAllowed = n - 1;
14    int maxOut = 0, maxIn = 0;
15    for (int d : finalOutDegrees) if (d > maxOut) maxOut = d;
16    for (int d : finalInDegrees) if (d > maxIn) maxIn = d;

```

```

17     if (maxOut > maxAllowed) {
18         double coeff = (double)maxAllowed / maxOut;
19         for (int i = 0; i < n; i++) {
20             finalOutDegrees[i] = (int)(finalOutDegrees[i] * coeff);
21             if (finalOutDegrees[i] < 1) finalOutDegrees[i] = 1;
22             if (finalOutDegrees[i] > maxAllowed) finalOutDegrees[i] =
maxAllowed;
23         }
24     }
25
26     if (maxIn > maxAllowed) {
27         double coeff = (double)maxAllowed / maxIn;
28         for (int i = 0; i < n; i++) {
29             finalInDegrees[i] = (int)(finalInDegrees[i] * coeff);
30             if (finalInDegrees[i] < 1) finalInDegrees[i] = 1;
31             if (finalInDegrees[i] > maxAllowed) finalInDegrees[i] = maxAllowed
;
32         }
33     }
34
35     for (int i = 0; i < n; i++) {
36         if (finalOutDegrees[i] < 1) finalOutDegrees[i] = 1;
37         if (finalInDegrees[i] < 1) finalInDegrees[i] = 1;
38         if (finalOutDegrees[i] > n - 1) finalOutDegrees[i] = n - 1;
39         if (finalInDegrees[i] > n - 1) finalInDegrees[i] = n - 1;
40     }
41
42     int outSum = 0, inSum = 0;
43     for (int d : finalOutDegrees) outSum += d;
44     for (int d : finalInDegrees) inSum += d;
45
46     if (outSum != inSum) {
47         int diff = outSum - inSum;
48         if (diff > 0) {
49             finalInDegrees[0] += diff;
50         } else {
51             finalOutDegrees[0] += -diff;
52         }
53     }
54
55     outDegrees = finalOutDegrees;
56     inDegrees = finalInDegrees;
57
58     for (auto& neighbors : adj)
59         neighbors.clear();
60
61     std::vector<std::pair<int, int>> arcs;
62
63     // список исходящих ребер для каждой вершины
64     for (int u = 0; u < n; u++) {
65         for (int i = 0; i < outDegrees[u]; i++) {
66             arcs.push_back({u, -1});

```

```

67     }
68 }
69
70 std::vector<int> inRemaining = inDegrees;
71
72 std::random_device rd;
73 std::mt19937 g(rd());
74
75 std::set<std::pair<int, int>> usedArcs;
76
77 for (auto& arc : arcs) {
78     int u = arc.first;
79
80     //только вершины v > u верхняя( треугольная матрица)
81     // и с ненулевой остаточной входящей степенью чтоб ребро еще не было
использовано
82     std::vector<int> candidates;
83     for (int v = u + 1; v < n; v++) {
84         if (inRemaining[v] > 0 && usedArcs.find({u, v}) == usedArcs.end())
85     {
86         candidates.push_back(v);
87     }
88
89     // если таких нет ищем среди всех но( все равно исключаем петли)
90     if (candidates.empty()) {
91         for (int v = u + 1; v < n; v++) {
92             if (v != u && usedArcs.find({u, v}) == usedArcs.end()) {
93                 candidates.push_back(v);
94             }
95         }
96     }
97
98     // если да так стрем берем любую v > u даже( если inRemaining[v] == 0)
99     if (candidates.empty()) {
100         for (int v = u + 1; v < n; v++) {
101             if (v != u) {
102                 candidates.push_back(v);
103             }
104         }
105     }
106
107     if (!candidates.empty()) {
108         std::uniform_int_distribution<int> dist(0, candidates.size() - 1);
109         int v = candidates[dist(g)];
110         arc.second = v;
111         usedArcs.insert({u, v}); // коллекция использованных ребрышек
112         if (inRemaining[v] > 0) {
113             inRemaining[v]--;
114         }
115     }
116 }

```

```

117
118 //строим список смежности
119 for (const auto& arc : arcs) {
120     if (arc.second != -1 && arc.first < arc.second) {
121         bool alreadyExists = false;
122         for (int existing : adj[arc.first]) {
123             if (existing == arc.second) {
124                 alreadyExists = true;
125                 break;
126             }
127         }
128         if (!alreadyExists) {
129             adj[arc.first].push_back(arc.second);
130         }
131     }
132 }
133
134 for (int i = 0; i < n; i++) {
135     outDegrees[i] = adj[i].size();
136 }
137
138 std::vector<int> actualInDegrees(n, 0);
139 for (int i = 0; i < n; i++) {
140     for (int v : adj[i]) {
141         actualInDegrees[v]++;
142     }
143 }
144 inDegrees = actualInDegrees;
145
146 return true;
147 }

```

2.5 Класс GraphAnalyzer

Класс GraphAnalyzer вычисляет характеристики графа: эксцентриситеты, центр, диаметр, а также реализует проверку эйлеровости и поиск эйлера цикла.

2.5.1 Вычисление эксцентриситетов

Функция computeAllDistances выполняет BFS из каждой вершины для вычисления матрицы кратчайших расстояний. Функция computeEccentricities на основе этой матрицы находит максимальное расстояние от каждой вершины.

Вход: *int n, const Graph& graph* - граф и количество вершин.

Выход: *std::vector< std::vector< int >> distances* - матрица кратчайших расстояний

```

1 std::vector<int> GraphAnalyzer::computeEccentricities() const

```

```

2 {
3     std::vector<int> ecc(n, 0);
4     for (int i = 0; i < n; i++)
5         for (int j = 0; j < n; j++)
6             if (distances[i][j] > ecc[i])
7                 ecc[i] = distances[i][j];
8     return ecc;
9 }
10
11 std::pair<int, std::vector<int>> GraphAnalyzer::findCenter() const
12 {
13     auto ecc = computeEccentricities();
14     int radius = *std::min_element(ecc.begin(), ecc.end());
15     std::vector<int> center;
16     for (int i = 0; i < n; i++)
17         if (ecc[i] == radius)
18             center.push_back(i);
19     return {radius, center};
20 }
21
22 std::pair<int, std::vector<int>> GraphAnalyzer::findDiameter() const
23 {
24     auto ecc = computeEccentricities();
25     int diameter = *std::max_element(ecc.begin(), ecc.end());
26     std::vector<int> diamVertices;
27     for (int i = 0; i < n; i++)
28         if (ecc[i] == diameter)
29             diamVertices.push_back(i);
30     return {diameter, diamVertices};
31 }
32

```

2.6 Класс WeightedGraph

Класс WeightedGraph хранит матрицу весов и реализует метод Шимбелла и алгоритм Дейкстры.

2.6.1 Генерация весовой матрицы

Функция generateWeightsFromGraph генерирует веса рёбер на основе структуры графа и логарифмического распределения. Пользователь выбирает знак весов: положительные, отрицательные или смешанные.

Вход: *const Graph& graph*, *LogDistribution& logDist*, *SignType signType*, *RandomGenerator& rng* - граф, логарифмическое распределение, тип знака, генератор случайных чисел

Выход: *const std::vector< std::vector< double >> & weightMatrix* - заполненная матрица весов

```

1 void WeightedGraph::generateWeightsFromGraph(

```

```

2   const Graph& graph, LogDistribution& logDist,
3   SignType signType, RandomGenerator& rng)
4   {
5   for (int u = 0; u < n; u++) {
6       for (int v : graph.getNeighbors(u)) {
7           int weight = logDist.generate();
8           weight = std::abs(weight) % 100;
9           if (weight == 0) weight = 1;
10          switch (signType) {
11              case POSITIVE:
12                  weight = std::abs(weight); break;
13              case NEGATIVE:
14                  weight = -std::abs(weight); break;
15              case MIXED:
16                  if (rng.uniformInt(0, 1) == 0)
17                      weight = std::abs(weight);
18                  else
19                      weight = -std::abs(weight);
20                  break;
21          }
22          weightMatrix[u][v] = weight;
23          edges.push_back(Edge(u, v, weight));
24          edgeCount++;
25          if (graph.getType() == Graph::UNDIRECTED)
26              weightMatrix[v][u] = weight;
27      }
28  }
29  }
30

```

2.6.2 Метод Шимбелла

Функция `findMinPaths` (`findMaxPaths`) реализует метод Шимбелла для нахождения минимальных (максимальных) путей заданной длины. Последовательно выполняется «умножение» матриц по правилам Шимбелла.

Вход: *intlength* - длина пути (количество рёбер)

Выход: *const std::vector< std::vector< double >> &prevDist* матрица минимальных (максимальных) путей

```

1   std::vector<std::vector<double>> WeightedGraph::findMinPaths(
2   int length) const
3   {
4       lastMinPathsIterations = 0;
5       if (length <= 1) return weightMatrix;
6
7       auto prevDist = weightMatrix;
8       for (int step = 2; step <= length; step++) {
9           lastMinPathsIterations++;
10          std::vector<std::vector<double>> newDist(n,
11          std::vector<double>(n, INF));

```



```

12     for (int i = 0; i < n; i++)
13         for (int j = 0; j < n; j++)
14             for (int k = 0; k < n; k++) {
15                 lastMinPathsIterations++;
16                 if (prevDist[i][k] < INF - 1
17                     && weightMatrix[k][j] < INF - 1) {
18                     double candidate =
19                         prevDist[i][k] + weightMatrix[k][j];
20                     if (candidate < newDist[i][j] - 1e-12)
21                         newDist[i][j] = candidate;
22                 }
23             }
24     prevDist = newDist;
25 }
26 return prevDist;
27 }
28

```

2.6.3 Алгоритм Дейкстры

Функция `dijkstra` реализует классический алгоритм Дейкстры. Проверяет наличие отрицательных весов и отказывает в работе при их обнаружении.

Вход: *intstart*, *intend* - начальная вершина, конечная вершина

Выход: *DijkstraResult(T, H, lastDijkstraIterations)* - вектор кратчайших расстояний *T*, вектор предшественников *H*, количество итераций

```

1  WeightedGraph::DijkstraResult WeightedGraph::dijkstra(
2      int start, int end) const
3  {
4      lastDijkstraIterations = 0;
5      std::vector<double> T(n, INF);
6      std::vector<int> H(n, -1);
7      std::vector<int> X(n, 0);
8      T[start] = 0;
9      X[start] = 1;
10     int current = start;
11
12     while (true) {
13         lastDijkstraIterations++;
14         for (int u = 0; u < n; u++) {
15             lastDijkstraIterations++;
16             if (weightMatrix[current][u] < INF - 1)
17                 if (X[u] == 0
18                     && T[u] > T[current]
19                     + weightMatrix[current][u]) {
20                     T[u] = T[current]
21                         + weightMatrix[current][u];
22                     H[u] = current;
23                 }
24         }
25     }
26 }

```

```

25
26     double m = INF;
27     int v = -1;
28     for (int u = 0; u < n; u++) {
29         lastDijkstraIterations++;
30         if (X[u] == 0 && T[u] < m) {
31             v = u; m = T[u];
32         }
33     }
34
35     if (v == -1) break;
36     if (v == end) { X[v] = 1; break; }
37     X[v] = 1;
38     current = v;
39 }
40 return DijkstraResult(T, H, lastDijkstraIterations);
41 }
42

```

2.7 Класс PathFinder

Класс PathFinder определяет возможность построения маршрута и подсчитывает количество простых путей между двумя вершинами.

Вход: *intcurrent, inttarget* - начальная вершина, конечная вершина

Выход: *std::vector<std::vector<int>>&allPaths* - список всех маршрутов

```

1 void PathFinder::findAllSimplePaths(
2     int current, int target,
3     std::vector<bool>& visited,
4     std::vector<int>& currentPath,
5     std::vector<std::vector<int>>& allPaths) const
6 {
7     currentPath.push_back(current);
8     if (current == target) {
9         allPaths.push_back(currentPath);
10        currentPath.pop_back();
11        return;
12    }
13    visited[current] = true;
14    for (int neighbor : graph.getNeighbors(current))
15        if (!visited[neighbor])
16            findAllSimplePaths(neighbor, target,
17                               visited, currentPath, allPaths);
18    visited[current] = false;
19    currentPath.pop_back();
20 }
21
22 int PathFinder::countAllSimplePaths(
23     int start, int end) const

```

```

24 {
25     if (start == end) return 1;
26     std::vector<bool> visited(n, false);
27     std::vector<int> currentPath;
28     std::vector<std::vector<int>> allPaths;
29     findAllSimplePaths(start, end, visited,
30         currentPath, allPaths);
31     return allPaths.size();
32 }
33

```

2.8 Класс MaxFlowGraph

Класс MaxFlowGraph реализует алгоритм Форда-Фалкерсона для нахождения максимального потока и алгоритм потока минимальной стоимости.

2.8.1 Алгоритм Форда-Фалкерсона

Функция computeMaxFlow использует алгоритм расстановки пометок для поиска увеличивающих путей. Множество S хранит достижимые вершины из s , множество N – просмотренные. Каждая помеченная вершина получает тройку (знак, предыдущая вершина, δ).

Вход: $std :: vector < int > S(n, 0), std :: vector < int > N(n, 0)$ $labels.assign(n, NodeLabel1, -1, 0)$ - достижимость вершины из s , просмотренность вершины матрица пропускных способностей, метки: направление, предыдущая вершина, величина изменения потока

Выход: $labels - std :: vector < NodeLabel > \&$ - вектор меток всех вершин

```

1 bool MaxFlowGraph::findAugmentingPath(
2     int s, int t,
3     std::vector<NodeLabel>& labels,
4     int& augmentDelta)
5 {
6     std::vector<int> S(n, 0);
7     std::vector<int> N(n, 0);
8     labels.assign(n, NodeLabel{1, -1, 0});
9     S[s] = 1;
10    labels[s] = {1, -1, INF};
11
12    while (true) {
13        int a = 0;
14        for (int v = 0; v < n; v++) {
15            if (S[v] == 1 && N[v] == 0) {
16                for (int u = 0; u < n; u++) {
17                    if (S[u] == 0
18                        && flow[v][u] < capacity[v][u]) {
19                        S[u] = 1;
20                        int newDelta = std::min(

```

```

21         labels[v].delta,
22         capacity[v][u] - flow[v][u]);
23     labels[u] = {1, v, newDelta};
24     a = 1;
25 }
26 }
27 for (int u = 0; u < n; u++) {
28     if (S[u] == 0 && flow[u][v] > 0) {
29         S[u] = 1;
30         int newDelta = std::min(
31             labels[v].delta, flow[u][v]);
32         labels[u] = {-1, v, newDelta};
33         a = 1;
34     }
35 }
36 N[v] = 1;
37 }
38 }
39 if (S[t] == 1) {
40     augmentDelta = labels[t].delta;
41     return true;
42 }
43 if (a == 0) break;
44 }
45 return false;
46 }
47
48 int MaxFlowGraph::fordFulkerson(int source, int sink)
49 {
50     computeMaxFlow(source, sink);
51     int maxFlow = 0;
52     for (int v = 0; v < n; v++)
53         maxFlow += flow[source][v];
54     return maxFlow;
55 }
56

```

2.8.2 Поток минимальной стоимости

Функция `minCostFlow` последовательно ищет кратчайший путь в остаточной сети с помощью алгоритма Дейкстры и направляет по нему поток.

Вход: *int*source, *intsink*, *intrequiredFlow* - источник, сток, требуемый поток ($= \lfloor 2/3 \cdot max \rfloor$)

Выход: *inttotalCost* - минимальная стоимость для заданного потока

```

1  int MaxFlowGraph::minCostFlow(
2      int source, int sink, int requiredFlow)
3  {
4      WeightedGraph tempGraph(n);
5      int totalCost = 0;

```

```

6      int currentFlow = 0;
7
8      while (currentFlow < requiredFlow) {
9          lastMinCostFlowIterations++;
10         for (int i = 0; i < n; i++)
11             for (int j = 0; j < n; j++) {
12                 int residualCap =
13                     capacity[i][j] - flow[i][j];
14                 if (residualCap > 0)
15                     tempGraph.setWeight(i, j, cost[i][j]);
16                 else
17                     tempGraph.setWeight(i, j,
18                         WeightedGraph::getInfinity());
19             }
20
21         auto result = tempGraph.dijkstra(source, sink);
22         if (result.distances[sink]
23             >= WeightedGraph::getInfinity() - 1)
24             break;
25
26         std::vector<int> path;
27         int current = sink;
28         while (current != -1) {
29             path.push_back(current);
30             current = result.previous[current];
31         }
32         std::reverse(path.begin(), path.end());
33
34         int pathFlow = requiredFlow - currentFlow;
35         for (size_t i = 0; i < path.size() - 1; i++) {
36             pathFlow = std::min(pathFlow,
37                 capacity[path[i]][path[i+1]]
38                 - flow[path[i]][path[i+1]]);
39         }
40
41         for (size_t i = 0; i < path.size() - 1; i++) {
42             flow[path[i]][path[i+1]] += pathFlow;
43             flow[path[i+1]][path[i]] -= pathFlow;
44             totalCost += pathFlow
45                 * cost[path[i]][path[i+1]];
46         }
47         currentFlow += pathFlow;
48     }
49     return totalCost;
50 }
51

```

2.9 Класс Skeleton

Класс Skeleton реализует алгоритмы для работы с остовными деревьями: матричная теорема Кирхгофа, алгоритм Прима, код Прюфера, минимальное рёберное покрытие и фундаментальная система разрезов.

2.9.1 Матричная теорема Кирхгофа

Функция countSpanningTrees вычисляет определитель минора матрицы Кирхгофа методом Гаусса, что даёт число остовных деревьев.

Вход: *std::vector<std::vector<int>> kirchhoff* - матрица Кирхгофа графа

Выход: *static_cast<int>(countSpanningTrees())* - количество остовных деревьев

```
1 long long Skeleton::countSpanningTrees() const
2 {
3     if (n <= 1) return 1;
4     std::vector<std::vector<double>> minor =
5         getMinor(kirchhoff, 0, 0);
6     return static_cast<long long>(
7         std::abs(determinant(minor)));
8 }
9
```

2.9.2 Алгоритм Прима

Функция primMST строит минимальное остовное дерево по алгоритму Прима.

Вход: *std::vector<std::vector<double>> weightMatrix* - матрица весов графа

Выход: *std::vector<Edge> T* - список рёбер минимального остова

```
1 std::vector<Skeleton::Edge> Skeleton::primMST() const
2 {
3     int p = n;
4     if (p == 0) return {};
5     int u = 0;
6     std::vector<bool> S(p, false);
7     S[u] = true;
8     std::vector<Edge> T;
9     std::vector<int> alpha(p, 0);
10    std::vector<double> beta(p,
11        WeightedGraph::getInfinity());
12
13    for (int v = 0; v < p; v++) {
14        if (v == u) continue;
15        if (weightMatrix[u][v]
16            < WeightedGraph::getInfinity() - 1) {
```

```

17         alpha[v] = u;
18         beta[v] = weightMatrix[u][v];
19     }
20 }
21
22 for (int i = 1; i <= p - 1; i++) {
23     double x = WeightedGraph::getInfinity();
24     int w = -1;
25     for (int v = 0; v < p; v++)
26         if (!S[v] && beta[v] < x) {
27             w = v; x = beta[v];
28         }
29     if (w == -1) break;
30     S[w] = true;
31     T.push_back(Edge(alpha[w], w,
32         weightMatrix[alpha[w]][w]));
33     for (int v = 0; v < p; v++)
34         if (weightMatrix[w][v]
35             < WeightedGraph::getInfinity() - 1)
36             if (!S[v] && beta[v] > weightMatrix[v][w]) {
37                 alpha[v] = w;
38                 beta[v] = weightMatrix[v][w];
39             }
40 }
41 return T;
42 }
43

```

2.9.3 Кодирование и декодирование Прюфера

Функция `encodePruferWithWeights` кодирует дерево в код Прюфера, сохраняя при этом веса рёбер.

Вход: *const std::vector< Edge > &tree* - список рёбер остовного дерева

Выход: *std::vector< int > A, std::vector< int > &outWeights* - код Прюфера и веса

```

1 std::vector<int> Skeleton::encodePruferWithWeights(
2     const std::vector<Edge>& tree,
3     std::vector<int>& outWeights) const
4 {
5     std::vector<std::set<int>> Gamma(n);
6     std::vector<int> degree(n, 0);
7     std::vector<std::vector<int>> edgeWeight(n,
8         std::vector<int>(n, 0));
9     for (const auto& e : tree) {
10         Gamma[e.from].insert(e.to);
11         Gamma[e.to].insert(e.from);
12         degree[e.from]++; degree[e.to]++;
13         edgeWeight[e.from][e.to] = (int)e.weight;
14         edgeWeight[e.to][e.from] = (int)e.weight;

```

```

15     }
16     std::vector<int> A;
17     outWeights.clear();
18     std::set<int> V;
19     for (int i = 0; i < n; i++) V.insert(i);
20     for (int i = 0; i < n - 2; i++) {
21         int v = -1;
22         for (int k : V)
23             if (degree[k] == 1) { v = k; break; }
24         int neighbor = *Gamma[v].begin();
25         A.push_back(neighbor);
26         outWeights.push_back(edgeWeight[v][neighbor]);
27         V.erase(v);
28         Gamma[neighbor].erase(v);
29         degree[v]--; degree[neighbor]--;
30     }
31     if (V.size() == 2) {
32         int a = *V.begin(), b = *V.rbegin();
33         outWeights.push_back(edgeWeight[a][b]);
34     }
35     return A;
36 }
37

```

Функция decodePrufer восстанавливает дерево из кода.

Вход: $std::vector<int> A, std::vector<int> \&outWeights$ - код Прюфера и веса

Выход: $std::vector<Edge> E$ - множество ребер

```

1  std::vector<Skeleton::Edge> Skeleton::decodePrufer(const std::vector<int>&
2  code, const std::vector<int>& weights, Edge* lastEdge = nullptr) const
3  {
4      // вход: массив A[1..p] - код Прюфера
5      // выход: множество ребер E
6
7      int p = code.size() + 2;
8      std::vector<Edge> E; //множество ребер
9      std::set<int> B; //множество вершин остова которые еще не были
10     использованы
11     for (int i = 0; i < p; i++) B.insert(i);
12
13     for (size_t i = 0; i < code.size(); i++) {
14         // v := min{k in B | forall j >= i (k != A[j])}
15         int v = -1;
16         for (int k : B) {
17             bool found = false;
18             for (size_t j = i; j < code.size(); j++) {
19                 if (code[j] == k) {
20                     found = true;
21                     break;
22                 }
23             }
24         }
25     }
26 }
27

```



```

22         if (!found) {
23             v = k; //значит это лист, супееер
24             break;
25         }
26     }
27
28     int weight = (i < weights.size()) ? weights[i] : 0;
29     E.push_back(Edge(v, code[i], weight));
30     B.erase(v);
31 }
32
33 // последнее ребро - оставшиеся две вершины
34 if (B.size() == 2) {
35     int u = *B.begin();
36     int v = *B.rbegin();
37     int lastWeight = (weights.size() > code.size()) ? weights.back() : 0;
38     Edge last(u, v, lastWeight);
39     E.push_back(last);
40
41     if (lastEdge != nullptr) {
42         *lastEdge = last;
43     }
44 }
45
46 return E;
47 }
48

```

2.9.4 Минимальное рёберное покрытие остова

Для остова используется жадный алгоритм, начинающий с листьев дерева.

Вход: *const std :: vector < Edge > &tree* - рёбра остова

Выход: *std :: vector < Edge > cover* - список рёбер минимального покрытия

```

1  std::vector<Skeleton::Edge> Skeleton::minEdgeCoverOnTree(const std::vector<
    Edge>& tree) const
2  {
3      // жадный алгоритмик для остовного дерева
4
5      std::vector<std::set<int>> adj(n);
6      std::vector<int> degree(n, 0);
7      std::vector<std::vector<double>> edgeW(n, std::vector<double>(n, -1));
8
9      // жесткое наполнение
10     for (const auto& e : tree) {
11         adj[e.from].insert(e.to);
12         adj[e.to].insert(e.from);
13         degree[e.from]++;

```

```

14         degree[e.to]++;
15         edgeW[e.from][e.to] = e.weight;
16         edgeW[e.to][e.from] = e.weight;
17     }
18
19     std::vector<bool> covered(n, false);
20     std::vector<bool> edgeInCover(n * n, false);
21     std::vector<Edge> cover;
22
23     // находим листья и родителя
24     std::queue<int> leaves;
25     for (int i = 0; i < n; i++) {
26         if (degree[i] == 1) leaves.push(i);
27     }
28
29     while (!leaves.empty()) {
30         int leaf = leaves.front();
31         leaves.pop();
32
33         if (degree[leaf] == 0) continue;
34
35         int parent = -1;
36         for (int v : adj[leaf]) {
37             if (degree[v] > 0) {
38                 parent = v;
39                 break;
40             }
41         }
42
43         if (parent != -1 && !covered[leaf]) {
44             // покрываем ребрышко
45             cover.push_back(Edge(leaf, parent, edgeW[leaf][parent]));
46             covered[leaf] = true;
47             covered[parent] = true;
48
49             // удаляем
50             adj[parent].erase(leaf);
51             degree[parent]--;
52             degree[leaf]--;
53
54             if (degree[parent] == 1) leaves.push(parent);
55         }
56     }
57
58     // проверка все ли покрыли
59     for (int i = 0; i < n; i++) {
60         if (!covered[i]) {
61             double minWeight = WeightedGraph::getInfinity();
62             int bestNeighbor = -1;
63
64             for (int j = 0; j < n; j++) {
65                 if (i != j && weightMatrix[i][j] < WeightedGraph::getInfinity

```

```

66         () - 1) {
67             if (weightMatrix[i][j] < minWeight) {
68                 minWeight = weightMatrix[i][j];
69                 bestNeighbor = j;
70             }
71         }
72
73         if (bestNeighbor != -1) {
74             cover.push_back(Edge(i, bestNeighbor, minWeight));
75             covered[i] = true;
76             covered[bestNeighbor] = true;
77         }
78     }
79 }
80
81
82 return cover;
83 }
84

```

2.9.5 Минимальное рёберное покрытие графа

Для остова используется жадный алгоритм, начинающий с листьев дерева. Для исходного графа строится максимальное паросочетание с приоритетом по степеням, затем покрываются оставшиеся вершины.

Вход: *std::vector<std::vector<double>> weightMatrix* - весовая матрица весов

Выход: *std::vector<Edge> cover* - список рёбер минимального покрытия

```

1 std::vector<Skeleton::Edge> Skeleton::minEdgeCoverOnGraph() const
2 {
3     std::vector<int> match(n, -1); //ну метч на то и метч что метч
4     std::vector<bool> used(n, false); //метки соединенных
5
6     // собираем вершины и сортируем по степени
7     std::vector<std::pair<int, int>> verticesByDegree; // {degree, vertex}
8     for (int i = 0; i < n; i++) {
9         int deg = 0;
10        for (int j = 0; j < n; j++) {
11            if (i != j && weightMatrix[i][j] < WeightedGraph::getInfinity() -
12        1) {
13                deg++;
14            }
15        }
16        verticesByDegree.push_back({deg, i});
17    }
18    std::sort(verticesByDegree.begin(), verticesByDegree.end());

```

```

19 // паросочетание с приоритетами по степеням связи
20 for (const auto& p : verticesByDegree) {
21     int i = p.second;
22     if (used[i]) continue;
23
24     // накидываем соседей
25     std::vector<std::pair<int, int>> freeNeighbors; // {degree, vertex}
26     for (int j = 0; j < n; j++) {
27         if (!used[j] && i != j && weightMatrix[i][j] < WeightedGraph::
getInfinity() - 1) {
28             int deg = 0;
29             for (int k = 0; k < n; k++) {
30                 if (k != j && weightMatrix[j][k] < WeightedGraph::
getInfinity() - 1) {
31                     deg++;
32                 }
33             }
34             freeNeighbors.push_back({deg, j});
35         }
36     }
37
38     if (!freeNeighbors.empty()) {
39         // выбираем минимального соседа
40         std::sort(freeNeighbors.begin(), freeNeighbors.end());
41         int j = freeNeighbors[0].second;
42
43         match[i] = j;
44         match[j] = i;
45         used[i] = true;
46         used[j] = true;
47     }
48 }
49
50 std::vector<Edge> cover;
51 std::vector<bool> covered(n, false);
52
53 //добавляем рёбра в покрытие
54 for (int i = 0; i < n; i++) {
55     if (match[i] != -1 && i < match[i]) {
56         cover.push_back(Edge(i, match[i], weightMatrix[i][match[i]]));
57         covered[i] = true;
58         covered[match[i]] = true;
59     }
60 }
61
62 // покрываем оставшиеся
63 std::vector<std::pair<int, int>> uncovered; // {degree, vertex}
64 for (int i = 0; i < n; i++) {
65     if (!covered[i]) {
66         int deg = 0;
67         for (int j = 0; j < n; j++) {
68             if (i != j && weightMatrix[i][j] < WeightedGraph::getInfinity

```

```

69         deg++;
70     }
71 }
72 uncovered.push_back({deg, i});
73 }
74 }
75 // приоритеты: МИН степень -> МИН вес
76 std::sort(uncovered.begin(), uncovered.end());
77 for (const auto& p : uncovered) {
78     int i = p.second;
79     if (!covered[i]) {
80         double minWeight = WeightedGraph::getInfinity();
81         int bestNeighbor = -1;
82
83         std::vector<std::pair<int, std::pair<double, int>>> candidates;
84         for (int j = 0; j < n; j++) {
85             if (i != j && weightMatrix[i][j] < WeightedGraph::getInfinity() - 1) {
86                 int deg = 0;
87                 for (int k = 0; k < n; k++) {
88                     if (k != j && weightMatrix[j][k] < WeightedGraph::getInfinity() - 1) {
89                         deg++;
90                     }
91                 }
92                 candidates.push_back({deg, {weightMatrix[i][j], j}});
93             }
94         }
95
96         if (!candidates.empty()) {
97             std::sort(candidates.begin(), candidates.end());
98             bestNeighbor = candidates[0].second.second;
99             minWeight = candidates[0].second.first;
100             cover.push_back(Edge(i, bestNeighbor, minWeight));
101             covered[i] = true;
102         }
103     }
104 }
105
106 return cover;
107 }

```

2.9.6 Фундаментальная система разрезов

Функция `getFundamentalCuts` для каждого ребра остова удаляет его и определяет две компоненты связности. Рёбра исходного графа между этими компонентами образуют фундаментальный разрез.

Вход: `std::vector<std::vector<double>> weightMatrix` - матрица весов графа

Выход: $std :: vector < Cut > fundamentalCuts$ - фундаментальная система разрезов

```
1 std::vector<Skeleton::Cut> Skeleton::getFundamentalCuts()
2 {
3     std::vector<Edge> mst = primMST();// остов
4     mstEdges = mst;
5
6     // список смежности остова
7     std::vector<std::vector<int>> treeAdj(n);
8     for (const auto& e : mst) {
9         treeAdj[e.from].push_back(e.to);
10        treeAdj[e.to].push_back(e.from);
11    }
12
13    std::vector<Cut> fundamentalCuts;
14
15    for (const auto& e : mst) {
16        int u = e.from;
17        int v = e.to;
18
19        // обходим дерево начиная с u
20        std::vector<bool> visited(n, false);
21        std::queue<int> q; // вершины связанные с u
22
23        q.push(u);
24        visited[u] = true;
25
26        while (!q.empty()) {
27            int cur = q.front();
28            q.pop();
29
30            for (int neighbor : treeAdj[cur]) {
31                // пропускаем переход через ребро (u,v)
32                if ((cur == u && neighbor == v) || (cur == v && neighbor == u)
33            ) {
34                continue;
35            }
36            if (!visited[neighbor]) {
37                visited[neighbor] = true;
38                q.push(neighbor);
39            }
40        }
41
42        // visited[i] == true → вершина в V1 достижима( из u без ребра (u,v))
43        // visited[i] == false → вершина в V2
44
45        Cut cut;
46
47        // сохраняем вершины в V1 и V2
48        for (int i = 0; i < n; i++) {
```

```

49         if (visited[i]) {
50             cut.vertices.push_back(i); // добавляем соединенные с u в v1
51         }
52         else cut.vertices2.push_back(i); // добавляем остальные в v2
53     }
54
55     // добавляем все рёбра между V1 и V2
56     for (int i = 0; i < n; i++) {
57         if (!visited[i]) continue; // i in V1
58
59         for (int j = 0; j < n; j++) {
60             if (visited[j]) continue; // j in V2
61
62             if (weightMatrix[i][j] < WeightedGraph::getInfinity() - 1) {
63                 int a = std::min(i, j);
64                 int b = std::max(i, j);
65                 cut.edges.push_back({a, b});
66             }
67         }
68     }
69
70     // удаляем дубликаты ребер
71     std::vector<std::pair<int, int>> uniqueEdges;
72     std::sort(cut.edges.begin(), cut.edges.end());
73     for (const auto& edge : cut.edges) {
74         if (uniqueEdges.empty() || uniqueEdges.back() != edge) {
75             uniqueEdges.push_back(edge);
76         }
77     }
78     cut.edges = uniqueEdges;
79
80     fundamentalCuts.push_back(cut);
81 }
82
83 return fundamentalCuts;
84 }

```

Функция `symmetricDifference` вычисляет симметрическую разность двух разрезов.

Вход: *const std::vector<int> &cutIndices* - индексы фундаментальных разрезов

Выход: *Cutresult* - разрез полученный через симметрическую разность

```

1 Skeleton::Cut Skeleton::symmetricDifferenceMultiple(const std::vector<int>&
  cutIndices)
2 {
3     auto allCuts = getFundamentalCuts();
4
5     if (cutIndices.empty()) {
6         return Cut();
7     }
8

```

```

9      // начинаем с первого разреза
10     Cut result = allCuts[cutIndices[0]];
11
12     // последовательно применяем симметрическую разность
13     for (size_t i = 1; i < cutIndices.size(); i++) {
14         result = symmetricDifference(result, allCuts[cutIndices[i]]);
15     }
16
17     return result;
18 }
19

```

2.10 Эйлеров цикл

Проверка эйлеровости и модификация графа реализованы в классах Graph и GraphAnalyzer.

Функция makeEulerian в классе Graph модифицирует неориентированный граф до эйлерового: попарно соединяет вершины нечётной степени рёбрами или удаляет рёбра, обеспечивая чётность всех степеней. Все изменения логируются.

Вход: *vector < vector < int >> adj* - список смежности Выход: *std::stringlog* - лог изменений, *vector < vector < int >> adj* - список смежности

```

1  std::string Graph::makeEulerian()
2  {
3      std::stringstream log;
4
5
6      if (type == DIRECTED) {
7          log << "ERROR: Directed graphs not supported.\n";
8          return log.str();
9      }
10
11     std::vector<int> originalDegrees(n, 0);
12     for (int i = 0; i < n; i++) {
13         originalDegrees[i] = adj[i].size();
14     }
15
16     log << "Original degrees:\n";
17     for (int i = 0; i < n; i++) {
18         log << "Vertex" << i << ": " << originalDegrees[i];
19         if (originalDegrees[i] % 2 != 0) log << " + ODD";
20         log << "\n";
21     }
22     log << "\n";
23
24     int edgesAdded = 0;
25     int edgesRemoved = 0;
26

```



```

27 // продолжаем пока есть нечётные вершины
28 int maxIterations = n * 2;
29 int iter = 0;
30
31 while (iter < maxIterations) {
32     iter++;
33
34     auto odd = getOddDegreeVertices();
35
36     if (odd.empty()) {
37         log << "All degrees even! Done in " << iter << " iteration(s).\n"
;
38         break;
39     }
40
41     if (odd.size() % 2 != 0) {
42         log << "ERROR:" << odd.size()
43 << " odd-degree vertices (odd number) - impossible by Handshaking
Lemma!\n";
44         break;
45     }
46
47     log << "Iteration " << iter << ": " << odd.size() << " odd vertices: [
";
48     for (size_t i = 0; i < odd.size(); i++) {
49         log << odd[i];
50         if (i < odd.size() - 1) log << ", ";
51     }
52     log << "]\n";
53
54     bool progress = false;
55
56     // перебираем пары
57     for (size_t i = 0; i < odd.size() && !progress; i++) {
58         int u = odd[i];
59         if (adj[u].size() % 2 == 0) continue; // уже чётная
60
61         for (size_t j = i + 1; j < odd.size() && !progress; j++) {
62             int v = odd[j];
63             if (adj[v].size() % 2 == 0) continue; // уже чётная
64
65             if (!hasEdge(u, v)) {
66                 // случай 1: ребра нет -> добавляем
67                 addEdge(u, v);
68                 edgesAdded++;
69                 progress = true;
70                 log << "Added edge (" << u << ", " << v << ")\n";
71             }
72             else if (adj[u].size() >= 2 && adj[v].size() >= 2) {
73                 // случай 2: ребро есть, но можно безопасно удалить
74                 removeEdge(u, v);
75                 edgesRemoved++;

```

```

76         progress = true;
77         log << "░░-░Removed░edge░(" << u << ",░" << v << ")\n";
78     }
79     else {
80         // случай 3: ребро есть, но удалять нельзя (deg=1)
81         int weak = (adj[u].size() <= adj[v].size()) ? u : v;
82         int strong = (weak == u) ? v : u;
83
84         log << "░░Edge░(" << u << ",░" << v
85         << ")░exists,░vertex░" << weak
86         << "░has░deg=" << adj[weak].size() << "░→░restructuring\n";
87
88         ;
89
90         // ищем две чётные соединенные вершины
91         for (int e1 = 0; e1 < n && !progress; e1++) {
92             if (e1 == u || e1 == v) continue;
93             if (adj[e1].size() < 2) continue;
94             if (adj[e1].size() % 2 != 0) continue;
95
96             for (int e2 = e1 + 1; e2 < n && !progress; e2++) {
97                 if (e2 == u || e2 == v) continue;
98                 if (adj[e2].size() < 2) continue;
99                 if (adj[e2].size() % 2 != 0) continue;
100
101                 if (!hasEdge(e1, e2)) continue;
102
103                 // логика:
104                 // удаление (u,v) - обе нечётные теряют по 1
105                 // удаление (e1,e2) - обе чётные теряют по 1
106                 // соединение weak с e1 и e2 - weak получает +2,
107                 e1 e2 получают +1
108
109                 removeEdge(u, v);
110                 removeEdge(e1, e2);
111                 edgesRemoved += 2;
112
113                 if (!hasEdge(weak, e1)) {
114                     addEdge(weak, e1);
115                     edgesAdded++;
116                 }
117                 if (!hasEdge(weak, e2)) {
118                     addEdge(weak, e2);
119                     edgesAdded++;
120                 }
121
122                 progress = true;
123                 log << "░░░░-░Removed░(" << u << ",░" << v << ")\n";
124
125                 log << "░░░░-░Removed░(" << e1 << ",░" << e2 << ")\n";
126                 log << "░░░░+░Connected░" << weak << "░to░" << e1
127                 << "░and░" << e2 << "\n";

```

```

123         log << "░░░░Effect:░" << weak << "░-1+2=+1,░" <<
strong
124         << "░-1\n";
125     }
126 }
127
128     if (!progress) {
129         log << "░░░░░No░suitable░even░pair░found\n";
130     }
131 }
132 }
133 }
134
135 if (!progress) {
136     log << "░░No░progress░possible░in░this░iteration.\n";
137
138     // для всех оставшихся нечётных - просто добавляем связи
139     auto finalOdd = getOddDegreeVertices();
140     if (finalOdd.size() >= 2) {
141         log << "░░Force-connecting░remaining░odd░vertices:\n";
142         for (size_t i = 0; i < finalOdd.size(); i += 2) {
143             if (i + 1 >= finalOdd.size()) break;
144             int u = finalOdd[i];
145             int v = finalOdd[i + 1];
146             if (!hasEdge(u, v)) {
147                 addEdge(u, v);
148                 edgesAdded++;
149                 log << "░░░░+░Added░(" << u << ",░" << v << ") \n";
150                 progress = true;
151             }
152         }
153     }
154
155     if (!progress) break;
156 }
157 }
158
159 log << "\nFinal░result:\n";
160 log << "░░Edges░added:░" << edgesAdded << "\n";
161 log << "░░Edges░removed:░" << edgesRemoved << "\n\n";
162
163 for (int i = 0; i < n; i++) {
164     log << "░░Vertex░" << i << "░deg=" << adj[i].size();
165     if (adj[i].size() % 2 != 0) log << "░+░ODD!";
166     if (adj[i].size() == 0 && originalDegrees[i] > 0) log << "░+░ISOLATED!";
167     log << "\n";
168 }
169
170 if (allDegreesEven()) {
171     log << "\n░SUCCESS:░Graph░is░Eulerian!\n";
172 } else {

```

```

173         log << "\nFAILED: Could not make all degrees even.\n";
174     }
175
176     return log.str();
177 }

```

Функция `findEulerianCycle` в классе `GraphAnalyzer` ищет эйлеров цикл с помощью алгоритма Хирхольцера.

Вход: *vector < vector < int >> adj* - список смежности

Выход: *std::vector < int > cycle* - последовательность вершин эйлерова цикла

```

1  std::vector<int> GraphAnalyzer::findEulerianCycle() const
2  {
3      // вход: эйлеров граф G(V,E), заданный списками смежности
4      // выход: последовательность вершин эйлерова цикла
5
6      // проверяем, что граф эйлеров
7      if (!isEulerian()) {
8          return {};
9      }
10
11     копия\ списка смежности
12     std::vector<std::vector<int>> Gamma(n);
13     for (int i = 0; i < n; i++) {
14         Gamma[i] = graph.getNeighbors(i);
15     }
16
17     // S := 0 стек для хранения вершин
18     std::stack<int> S;
19
20     // select v in V произвольная вершина
21     int v = 0;
22     for (int i = 0; i < n; i++) {
23         if (!Gamma[i].empty()) {
24             v = i;
25             break;
26         }
27     }
28
29     // положить v в стек S
30     S.push(v);
31
32     std::vector<int> cycle;
33
34     // while S != 0 do
35     while (!S.empty()) {
36         // v := top S
37         v = S.top();
38
39         // if Γ[v] = 0 then
40         if (Gamma[v].empty()) {

```

```

41         // очередная вершина эйлера цикла
42         S.pop();
43         cycle.push_back(v);
44     } else {
45         // взять первую вершину из списка смежности
46         int u = Gamma[v].back();
47         // положить u в стек
48         S.push(u);
49         // удалить ребро (v,u)
50         Gamma[v].pop_back();
51         for (auto it = Gamma[u].begin(); it != Gamma[u].end(); ++it) {
52             if (*it == v) {
53                 Gamma[u].erase(it);
54                 break;
55             }
56         }
57     }
58 }
59
60 return cycle;
61 }
62

```

2.11 Функция main и графический интерфейс

Вкладки интерфейса:

1. Генерация графа и характеристики (эксцентриситеты, центр, диаметр)
2. Весовая матрица (генерация с выбором знака)
3. Метод Шимбелла (минимальные и максимальные пути)
4. Поиск маршрутов (количество путей между вершинами)
5. Алгоритмы (DFS, Дейкстра, сравнение итераций)
6. Потoki (Форд-Фалкерсон, поток минимальной стоимости)
7. Остовные деревья (Кирхгоф, Прима, Прюфер, рёберное покрытие)
8. Эйлеров цикл (проверка, модификация, построение)
9. Разрезы (фундаментальная система, симметрическая разность)

3 Результаты работы программы

Программа запускается с графическим интерфейсом Qt. Интерфейс состоит из 9 вкладок, каждая из которых соответствует определённому алгоритму или группе алгоритмов. Ниже описаны результаты работы программы для каждой вкладки.

3.1 Лабораторная работа №1

3.1.1 Генерация графа и характеристики

После запуска программы пользователь на первой вкладке задаёт количество вершин (например, 6), выбирает тип графа (ориентированный или неориентированный) и нажимает кнопку «Generate Graph». Программа формирует связный ациклический граф с использованием логарифмического распределения (параметр $p = 0.5$). Генерация происходит по степеням вершин: сначала для каждой вершины генерируется желаемая степень из логарифмического распределения, затем граф строится методом «stub matching» с проверкой связности.

В поле вывода отображается список смежности графа и степени вершин. Пример вывода для 6 вершин представлен на рисунке 14.

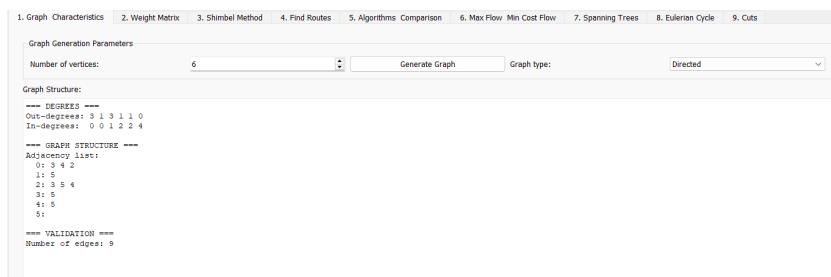


Рис. 14: Список смежности графа и степеней вершин

При нажатии кнопки «Show Characteristics» вычисляются и отображаются эксцентриситеты всех вершин, центр графа, диаметральные вершины и диаметральная цепь. Пример вывода представлен на рисунке 15.

Graph Characteristics:

Vertex eccentricities:

$e(0) = 2$
 $e(1) = 1$
 $e(2) = 1$
 $e(3) = 1$
 $e(4) = 1$
 $e(5) = 0$

Graph radius: 0

Center vertices: 5

Graph diameter: 2

Diametral vertices: 0

Diameter path: $0 \rightarrow 3 \rightarrow 5$

Рис. 15: Вывод характеристик графа

3.1.2 Генерация весовой матрицы

На второй вкладке пользователь выбирает тип весов (Positive / Negative / Mixed) и нажимает кнопку генерации. Весовая матрица генерируется на основе логарифмического распределения и отображается в виде таблицы QTableWidgetItem. Значение «—» означает отсутствие ребра (бесконечность). Диагональные элементы равны 0. Пример для положительных весов:

Weight Matrix Generation

Sign type: Mixed ▼

Only Positive
Only Negative
Mixed

Generate Weight Matrix

Edge Weights Matrix:

	0	1	2	3	4	5
0	0	—	1	-8	-3	—
1	—	0	—	—	—	1
2	—	—	0	7	-1	-8
3	—	—	—	0	—	-6
4	—	—	—	—	0	3
5	—	—	—	—	—	0

Рис. 16: Отображение весовой матрицы

3.1.3 Метод Шимбелла

На третьей вкладке пользователь задаёт длину пути (количество рёбер) с помощью слайдера (от 1 до количества вершин). При нажатии кнопки «Apply Shimbel» программа вычисляет и отображает две матрицы: минимальных и максимальных путей указанной длины. Пример для длины 2 изображён на рисунке 17

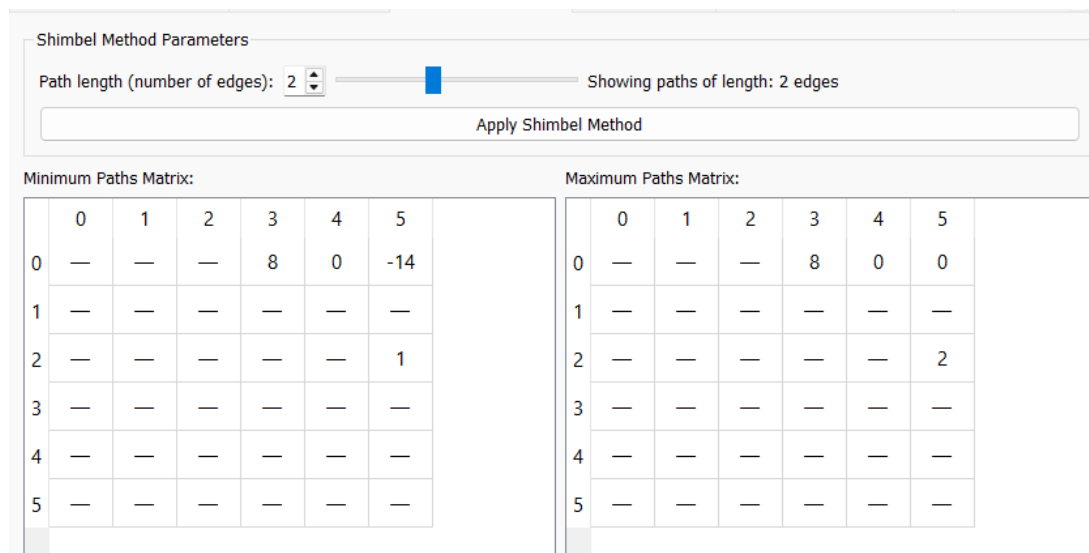


Рис. 17: Матрицы минимальных и максимальных путей указанной длины

3.1.4 Поиск маршрутов

На четвёртой вкладке пользователь вводит начальную и конечную вершины. Программа определяет возможность построения маршрута и выводит количество простых путей, а также перечисляет все найденные маршруты. Пример отображен на рисунке 18

Start vertex: 0 End vertex: 5 Find Routes

Routes from 0 to 5:

Total: 5 route(s)

1. 0 → 3 → 5
2. 0 → 4 → 5
3. 0 → 2 → 3 → 5
4. 0 → 2 → 5
5. 0 → 2 → 4 → 5

Рис. 18: Поиск маршрутов между указанными точками

Если маршрут невозможен, выводится сообщение, показанное на рисунке



Рис. 19: Вывод об отсутствии маршрутов

3.2 Лабораторная работа №2

3.2.1 Обход в глубину (DFS)

На пятой вкладке при нажатии кнопки «Perform DFS» программа выполняет обход графа поиском в глубину из заданной стартовой вершины. Результат содержит порядок обхода вершин и количество итераций алгоритма. Пример указан на рисунке 20

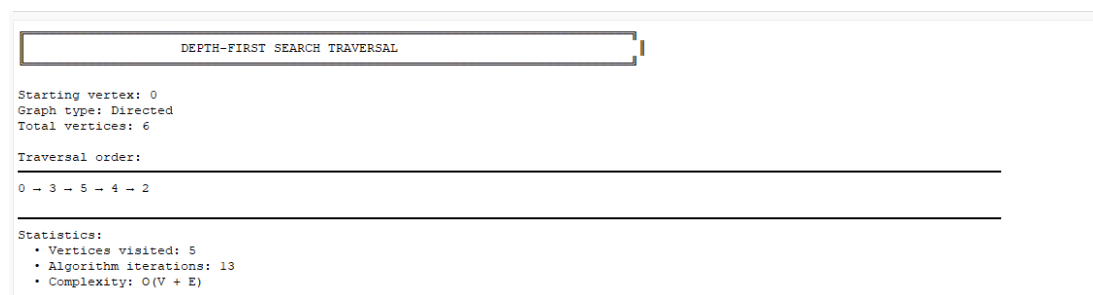


Рис. 20: Обход вершин графа поиском в глубину

3.2.2 Алгоритм Дейкстры

При нажатии кнопки «Perform Dijkstra» программа находит кратчайший путь между двумя вершинами. Выводится вектор расстояний T , вектор предшественников H , а также сам путь и его вес. Если матрица содержит отрицательные веса, выводится сообщение об ошибке и автоматически выполняется генерация весовой матрицы с положительными весами. Пример выполнения алгоритма с положительной весовой матрицей представлен на рисунке 21. Пример выполнения с отрицательной весовой матрицей представлен на рисунке 22.

```

=== DIJKSTRA ALGORITHM ===
Start vertex: 0
End vertex: 5

Distance vector T[v] (shortest path from 0 to v):
Vertex | Distance
-----|-----
0 | 0
1 | -
2 | 2
3 | 1
4 | 3
5 | 3

Predecessor vector H[v] (vertex preceding v on the shortest path):
Vertex | Predecessor
-----|-----
0 | -
1 | -
2 | 0
3 | 0
4 | 2
5 | 3

Path from 0 to 5:
Distance: 3
Route: 0 -> 3 -> 5

Algorithm iterations (outer loop): 52
Theoretical complexity:  $O(P^2) = 6^2 = 36$  operations

```

Рис. 21: Нахождение кратчайшего пути с использованием алгоритма Дейкстры

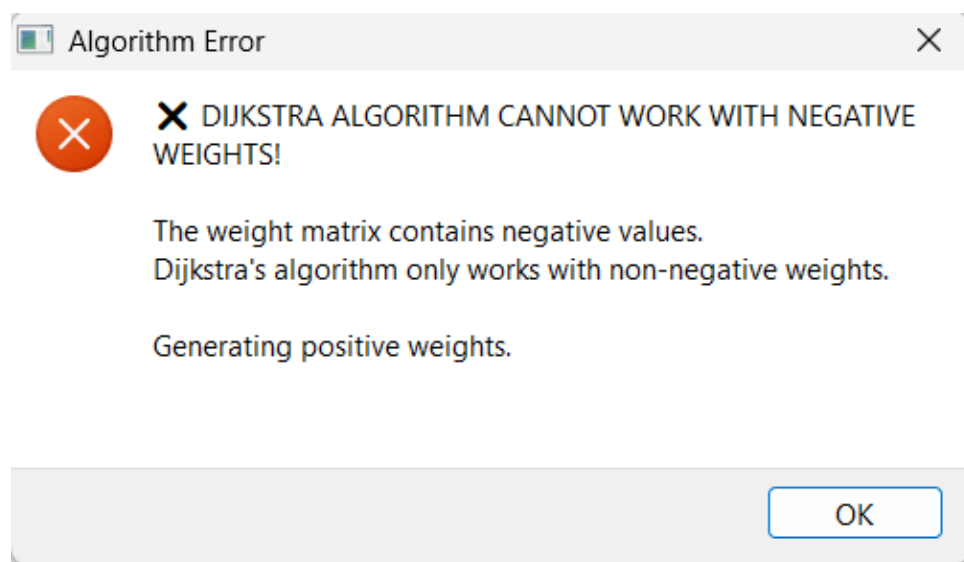


Рис. 22: Обработка случая вызова алгоритма Дейкстры с отрицательной весовой матрицей

3.2.3 Сравнение скоростей реализованных алгоритмов

При нажатии кнопки «Compare Algorithms» программа выполняет оба алгоритма для одних и тех же вершин и выводит количество итераций каждого. Пример вывода представлен на рисунке 23.

ALGORITHM PERFORMANCE COMPARISON	
Graph parameters: • Vertices: 6 • Type: Directed • Edges: 9	
ALGORITHM COMPARISON (by number of iterations)	
1. DEPTH-FIRST SEARCH (DFS) TRAVERSAL	
Iterations: 10 Theoretical complexity: $O(V + E)$ For current graph: $6 + 9 = 15$ operations	
2. DIJKSTRA ALGORITHM (shortest path from 0 to 5)	
Iterations: 52 Theoretical complexity: $O(V^2)$ For current graph: $6^2 = 36$ operations	

Рис. 23: Сравнение скоростей реализованных алгоритмов

3.3 Лабораторная работа №3

3.3.1 Максимальный поток (Форд-Фалкерсон)

На шестой вкладке при нажатии «Generate Capacities» генерируются матрицы пропускных способностей и стоимостей на основе логарифмического распределения. Матрицы отображаются в виде таблиц. Пример представлен на рисунке 24.

Flow Network Generation

Source vertex: 0 Sink vertex: 5

Generate Capacities and Costs

Flow Algorithms

Find Maximum Flow (Ford-Fulkerson) Find Minimum Cost Flow (2/3 of max)

Capacity Matrix:

	0	1	2	3	4	5
0	0	0	2	4	3	0
1	0	0	0	0	0	3
2	0	0	0	3	2	20
3	0	0	0	0	0	2
4	0	0	0	0	0	2
5	0	0	0	0	0	0

Cost Matrix:

	0	1	2	3	4	5
0	0	0	1	5	1	0
1	0	0	0	0	0	3
2	0	0	0	1	1	2
3	0	0	0	0	0	2
4	0	0	0	0	0	1
5	0	0	0	0	0	0

Рис. 24: Матрицы пропускных способностей и стоимостей

Далее при нажатии «Find Max Flow» вычисляется максимальный поток. Пример представлен на рисунке 25.

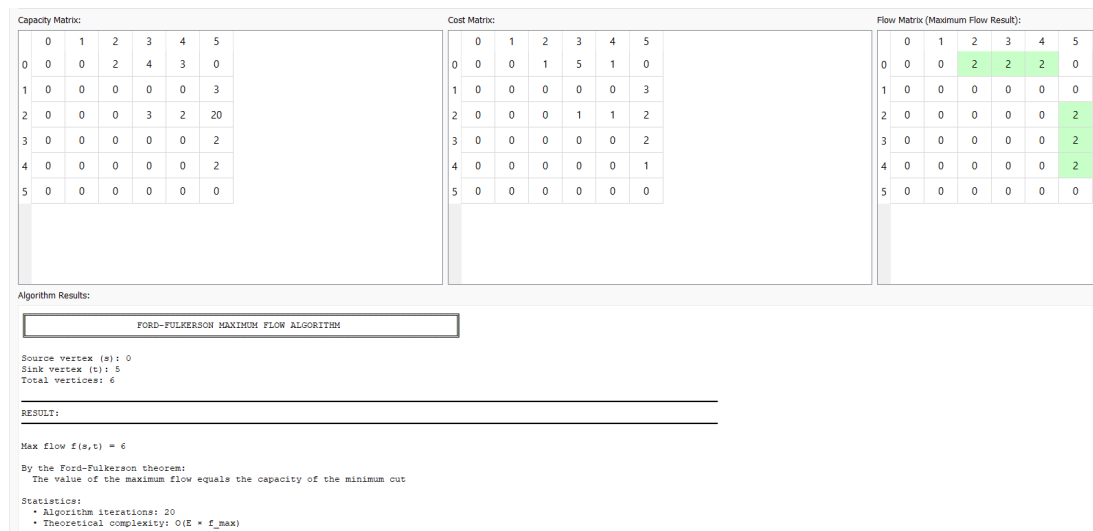


Рис. 25: Вычисление максимального потока

3.3.2 Поток минимальной стоимости

При нажатии «Find Min Cost Flow» вычисляется поток минимальной стоимости. Величина потока берётся как $\lfloor 2/3 \cdot max \rfloor$. Программа последовательно находит кратчайшие по стоимости пути алгоритмом Дейкстры на остаточной сети. Пример представлен на рисунке 26.

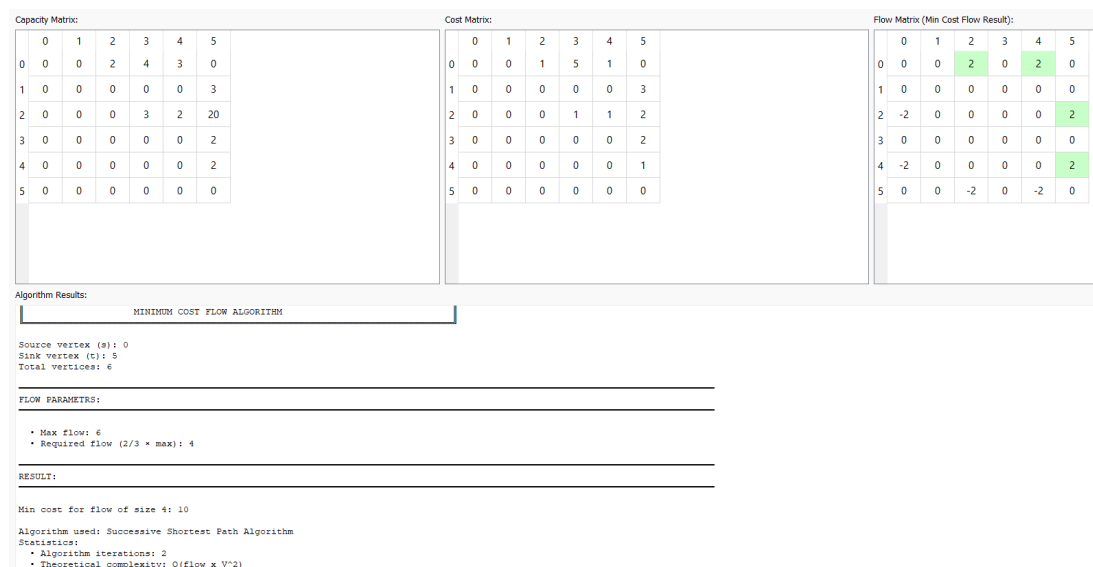


Рис. 26: Вычисление потока минимальной стоимости

3.4 Лабораторная работа №4

3.4.1 Число остовных деревьев (Кирхгоф)

На седьмой вкладке при нажатии «Count Spanning Trees» вычисляется матрица Кирхгофа, которая отображается в таблице. Затем вычисляется

определитель минора и выводится число остовных деревьев. Пример выполнения представлен на рисунке 27.

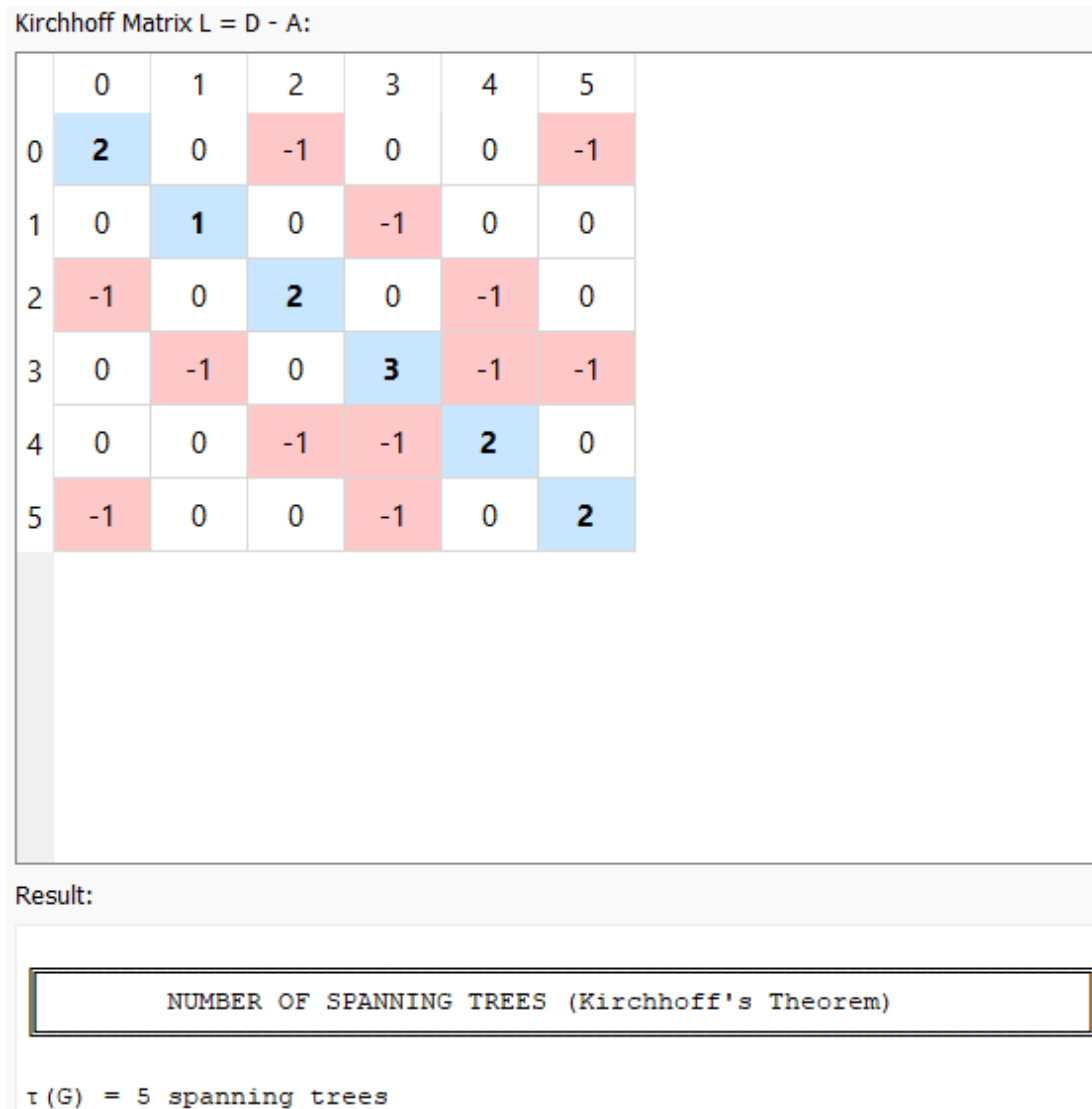


Рис. 27: Матрица Кирхгофа

3.4.2 Минимальный по весу остов (алгоритм Прима)

Строится минимальное остовное дерево. Выводятся рёбра остова и суммарный вес. Пример представлен на рисунке 28.

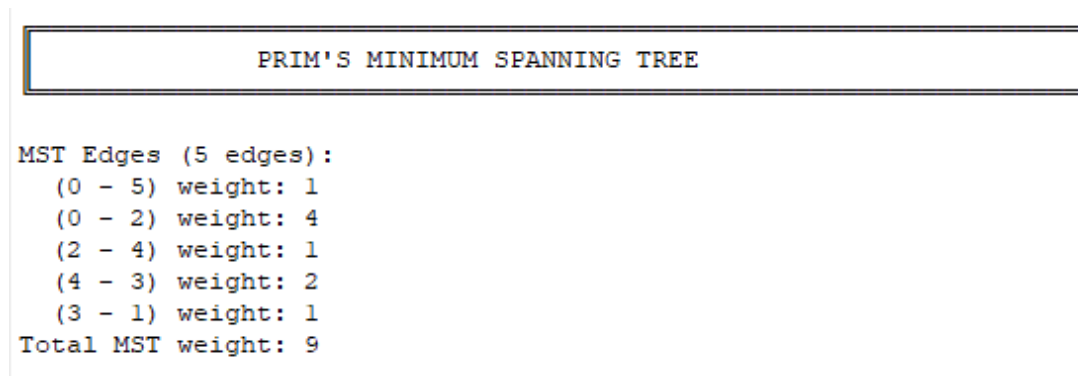


Рис. 28: Построение минимального остова

При попытке построить остов для ориентированного графа отображается предупреждение, представленное на рисунке 29.

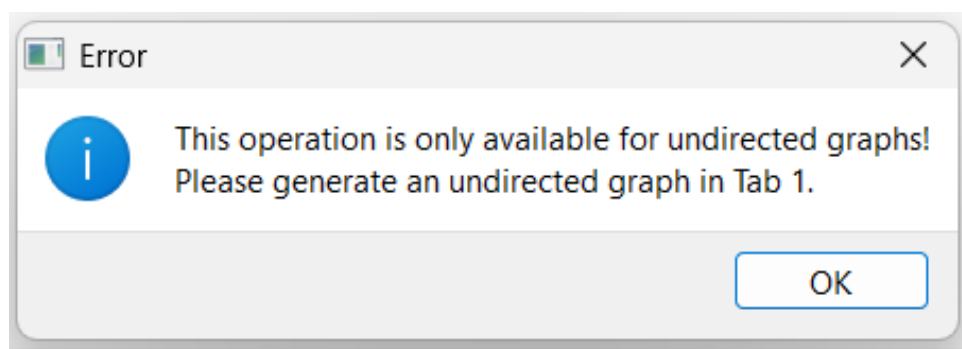


Рис. 29: Обработка случая генерации остова для ориентированного графа

3.4.3 Код Прюфера

Полученный остов кодируется кодом Прюфера с сохранением весов рёбер. Затем выполняется декодирование и проверяется совпадение с исходным деревом. Пример представлен на рисунке 30.

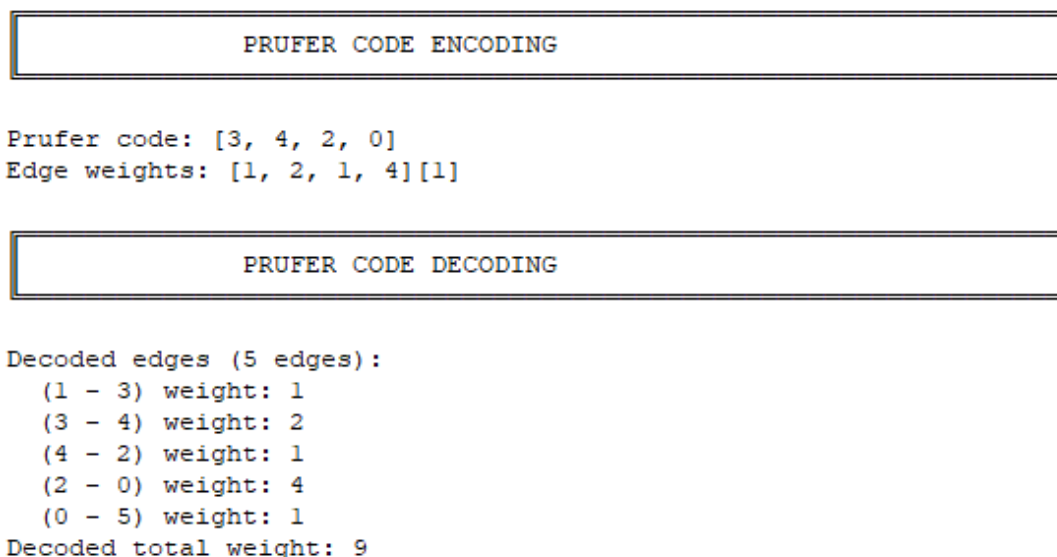


Рис. 30: Кодирование и декодирование Прюфера

3.4.4 Минимальное рёберное покрытие

Находится минимальное рёберное покрытие на остове и на исходном графе. Для каждого варианта выводятся рёбра покрытия, их количество и суммарный вес. Пример минимального реберного покрытия на остове представлен на рисунке 31, на изначальном графе - на рисунке 32

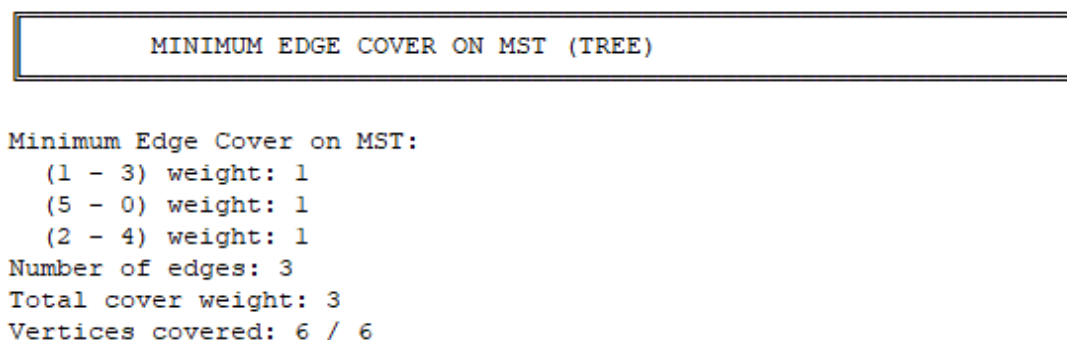


Рис. 31: Минимальное реберное покрытие на остове

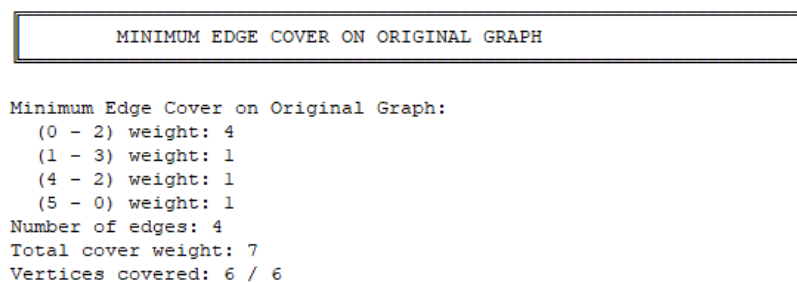


Рис. 32: Минимальное реберное покрытие на изначальном графе

3.5 Лабораторная работа №5

3.5.1 Проверка эйлеровости и построение эйлерова цикла

На восьмой вкладке при нажатии «Check Eulerian» проверяется, является ли неориентированный граф эйлеровым. Выводятся степени всех вершин и общее число рёбер. Если не все степени чётные, граф не является эйлеровым и выводится список вершин с нечётными степенями. Пример представлен на рисунке 33

```
Result:


EULERIAN CYCLE - FLEURY'S ALGORITHM


Graph type: Undirected
Number of vertices: 6

Vertex degrees:
deg(0) = 2
deg(1) = 1 ← ODD!
deg(2) = 2
deg(3) = 3 ← ODD!
deg(4) = 2
deg(5) = 2

Total edges: 6

✗ CONDITION FAILED: Not all vertices have even degree!
Vertices with odd degree (2): 1 3

→ To make the graph Eulerian, connect odd-degree vertices in pairs
or add edges to make all degrees even.
✓ CONDITION 2 MET: Graph is connected.



---


THE GRAPH IS NOT EULERIAN


---



To make it Eulerian, use the 'Modify Graph' function.
```

Рис. 33: Проверка графа на эйлеровость

При нажатии «Modify to Eulerian» граф модифицируется. Все изменения логируются: добавленные и удалённые рёбра, а также итоговые степени вершин. Пример представлен на рисунке 34.


```
Result:

MODIFYING GRAPH TO EULERIAN

Original degrees:
Vertex 0: 2
Vertex 1: 1 ← ODD
Vertex 2: 2
Vertex 3: 3 ← ODD
Vertex 4: 2
Vertex 5: 2

Iteration 1: 2 odd vertices: [1, 3]
Edge (1, 3) exists, vertex 1 has deg=1 → restructuring
- Removed (1, 3)
- Removed (0, 2) [even pair]
+ Connected 1 to 0 and 2
Effect: 1: -1+2=+1, 3: -1
✓ All degrees even! Done in 2 iteration(s).

Final result:
Edges added: 2
Edges removed: 2

Vertex 0: deg=2
Vertex 1: deg=2
Vertex 2: deg=2
Vertex 3: deg=2
Vertex 4: deg=2
Vertex 5: deg=2

✓ SUCCESS: Graph is Eulerian!
```

Рис. 34: Модификация графа до эйлера

После успешной модификации строится эйлеров цикл. Выводится последовательность вершин цикла, длина цикла и проверка корректности. Пример представлен на рисунке 35.

```
EULERIAN CYCLE - FLEURY'S ALGORITHM

Graph type: Undirected
Number of vertices: 6

Vertex degrees:
deg(0) = 2
deg(1) = 2
deg(2) = 2
deg(3) = 2
deg(4) = 2
deg(5) = 2

Total edges: 6

✓ CONDITION 1 MET: All vertices have even degree.
✓ CONDITION 2 MET: Graph is connected.

THE GRAPH IS EULERIAN!

Fleury's Algorithm Execution:
Using stack to track current path
Removing edges as they are traversed

EULERIAN CYCLE:
Start vertex: 0
Cycle length: 6 edges (7 vertices)

0 - 5 - 3 - 4 - 2 - 1 - 0

Verification:
✓ Starts and ends at vertex 0
✓ Uses exactly 6 edges (once each)
✓ All vertices visited appropriately
✓ Forms a valid cycle
```

Рис. 35: Проверка графа на эйлеровость и вывод эйлерового цикла

3.5.2 Фундаментальная система разрезов

На девятой вкладке строится фундаментальная система разрезов. Для каждого ребра остова выводятся множества вершин V_1 и V_2 , а также рёбра разреза. Пример представлен на рисунке 36.

```

FUNDAMENTAL CUT SYSTEM

Minimum Spanning Tree T (5 edges):
(0, 5)
(0, 2)
(2, 4)
(4, 3)
(3, 1)

Cocycloomatic number  $m^*(G) = 5$ 

FUNDAMENTAL CUT SYSTEM  $\delta = \{S_e\}_{e \in T}$ 

S_0 (removing edge 0 of T):
V1 = {0, 1, 2, 3, 4}
V2 = {5}
Edges in cut S (2 edges):
{(0, 5), (3, 5)}

S_1 (removing edge 1 of T):
V1 = {0, 5}
V2 = {1, 2, 3, 4}
Edges in cut S (2 edges):
{(0, 2), (3, 5)}

S_2 (removing edge 2 of T):
V1 = {0, 2, 5}
V2 = {1, 3, 4}
Edges in cut S (2 edges):
{(2, 4), (3, 5)}

S_3 (removing edge 3 of T):
V1 = {0, 2, 4, 5}
V2 = {1, 3}
Edges in cut S (2 edges):
{(3, 4), (3, 5)}

S_4 (removing edge 4 of T):
V1 = {0, 2, 3, 4, 5}
V2 = {1}
Edges in cut S (1 edges):
{(1, 3)}

```

Рис. 36: Фундаментальная система разрезов

Пользователь может выбрать несколько фундаментальных разрезов (по номерам) и получить их симметрическую разность. Примеры представлены на рисунке 37 и рисунке 38.

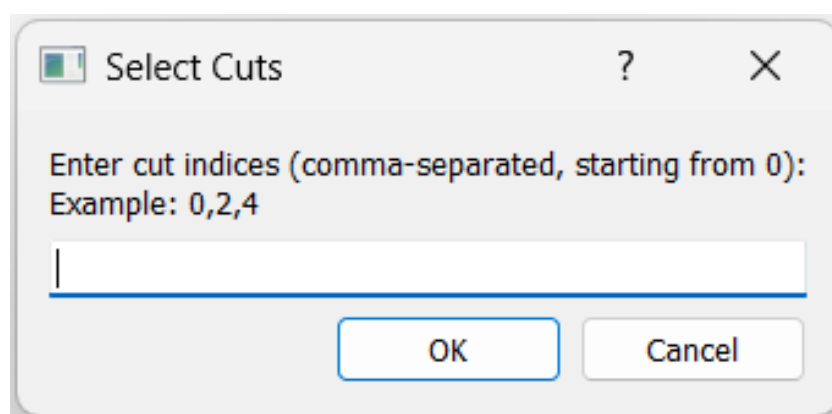


Рис. 37: Выбор фундаментальных разрезов для симметрической разности

SYMMETRIC DIFFERENCE OF MULTIPLE CUTS

Cuts: $S_0 \oplus S_1 \oplus S_3$

$V1 = \{0, 1, 3\}$

$V2 = \{\}$

Edges in cut S (4 edges):

$\{(0, 2), (0, 5), (3, 4), (3, 5)\}$

Рис. 38: Симметрическая разность выбранных разрезов

Заключение

По результатам выполнения лабораторных работ можно сделать следующие выводы:

В первой работе выполнены задачи по созданию случайного связного ациклического графа с использованием логарифмического распределения из справочника Вадзинского. Реализована генерация графов по степеням вершин. Вычислены эксцентриситеты всех вершин, выделен центр графа и диаметральные вершины. Реализован метод Шимбелла для нахождения минимальных и максимальных путей заданной длины. Реализовано определение возможности построения маршрута и подсчёт количества маршрутов.

Во второй работе реализован обход графа поиском в глубину (DFS) и классический алгоритм Дейкстры для нахождения кратчайшего пути. Алгоритм Дейкстры выводит вектор расстояний и путь в виде последовательности вершин. Проведено сравнение скоростей работы обоих алгоритмов по количеству итераций.

В третьей работе на основе ориентированного графа сгенерированы матрицы пропускных способностей и стоимости с помощью логарифмического распределения. Реализован алгоритм Форда-Фалкерсона для нахождения максимального потока и алгоритм потока минимальной стоимости с использованием алгоритма Дейкстры на остаточной сети.

В четвёртой работе найдено число остовных деревьев с использованием матричной теоремы Кирхгофа. Построен минимальный по весу остов алгоритмом Прима. Выполнено кодирование и декодирование остова с помощью кода Прюфера с сохранением весов рёбер. Реализован алгоритм нахождения минимального рёберного покрытия. Реализована фундаментальная система разрезов и операция симметрической разности.

В пятой работе проведена проверка графа на эйлеровость. Реализована модификация графа до эйлерового с логированием всех изменений. Построен эйлеров цикл. Повторно реализована фундаментальная система разрезов на основе минимального остова из четвёртой работы.

Достоинствами программы являются:

- Графический интерфейс пользователя на Qt со вкладками: всего 10 вкладок, каждая реализует отдельный функциональный блок;
- Модульная архитектура с разделением кода на классы;
- Использование контейнеров STL, обеспечивающих отсутствие утечек памяти.
- Нормирование степеней связи вершин при генерации для соответствия заданному распределению.

- Автоматическая генерация положительной весовой матрицы в случае отсутствия таковой в функциях, использующих алгоритм Дейкстры.

Недостатки и ограничения:

- Пользователь не может задать параметр распределения.
- В методе Шимбелла на каждом шаге создается полная новая матрица, для длины пути k выходит k полных матриц.

Возможности для масштабирования:

- Добавление визуального отображения сгенерированного графа и минимального остова.
- Возможна замена `vector` на `priority_queue` для потенциального ускорения алгоритма Дейкстры.
- Добавление возможности раннего выхода в алгоритме Дейкстры.
- Для метода Шимбелла использовать две матрицы как буферы и менять их местами.

Список литературы

- [1] Секция «Телематика» / текст : электронный / - URL: <https://tema.spbstu.ru/dismath/> (Дата обращения 18.05.2026).
- [2] Новиков Ф. А. Дискретная математика для программистов. 3-е изд. - Санкт-Петербург: Питер Пресс, 2009. - 384 с.
- [3] Вадзинский Р. Н. Справочник по вероятностным распределениям - Санкт-Петербург: Наука, 2001 - 295 с.
- [4] Кормен Т., Лейзер Ч. Алгоритмы. Построение и анализ - Москва; Вильямс, 2005 - 893 с.
- [5] Эриксон Дж. Алгоритмы - Москва; ДМК Пресс - 526 с.